# Application of Bayesian Inference to Reinforcement Learning

Information Theory and Inference project presentation

A.Y. 2025-26

Brocco Luca, Cafagno Samuele, Cavallin Jonathan

Università degli Studi di Padova

# Project introduction: the idea



**Reinforcement learning** is a collection of ML techniques that aim to solve non supervised problems

# Project introduction: the idea



**Reinforcement learning** is a collection of ML techniques that aim to solve non supervised problems

With RL is possible to solve problems by gradually **collecting datapoints** by exploring the environment

# Project introduction: the idea



**Reinforcement learning** is a collection of ML techniques that aim to solve non supervised problems

With RL is possible to solve problems by gradually **collecting datapoints** by exploring the environment

# Project introduction: the idea



**Reinforcement learning** is a collection of ML techniques that aim to solve non supervised problems

With RL is possible to solve problems by gradually **collecting datapoints** by exploring the environment

This is very **computationally expensive**, and requires many iterations of the training loops

# Project introduction: the idea



**Reinforcement learning** is a collection of ML techniques that aim to solve non supervised problems

With RL is possible to solve problems by gradually **collecting datapoints** by exploring the environment

This is very **computationally expensive**, and requires many iterations of the training loops

We try to implement **Bayesian Inference** approaches to provide a more efficient alternative to classical RL [2]

# Project introduction: RL basics



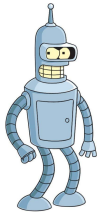
The **Agent** is the entity that aims to solve the task

# Project introduction: RL basics



The **Agent** is the entity that aims to solve the task

agent



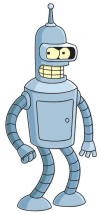
# Project introduction: RL basics



The **Agent** is the entity that aims to solve the task

The **State** is the situation in which the Agent finds itself at time t: it contains all the information about what the Agent knows

agent

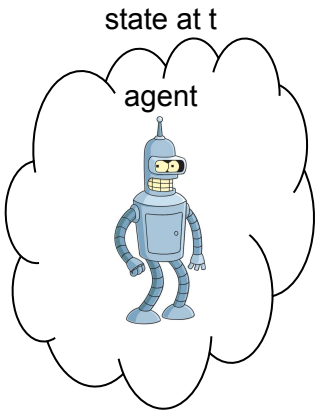


# Project introduction: RL basics



The **Agent** is the entity that aims to solve the task

The **State** is the situation in which the Agent finds itself at time t: it contains all the information about what the Agent knows



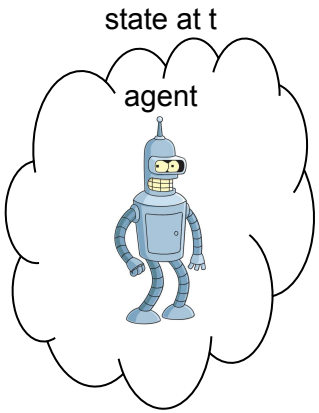
# Project introduction: RL basics



The **Agent** is the entity that aims to solve the task

The **State** is the situation in which the Agent finds itself at time t: it contains all the information about what the Agent knows

The **Environment** is everything that surrounds the agent. Based on agent's **Actions**, it provides **Rewards**



# Project introduction: RL basics

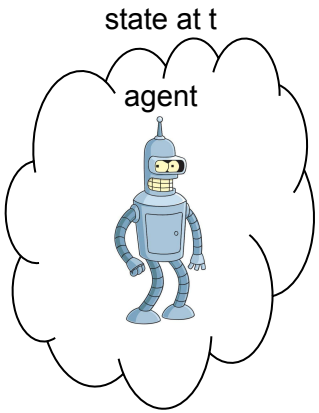


The **Agent** is the entity that aims to solve the task

The **State** is the situation in which the Agent finds itself at time t: it contains all the information about what the Agent knows

The **Environment** is everything that surrounds the agent. Based on agent's **Actions**, it provides **Rewards**

Learning happens in a feedback loop:



# Project introduction: RL basics

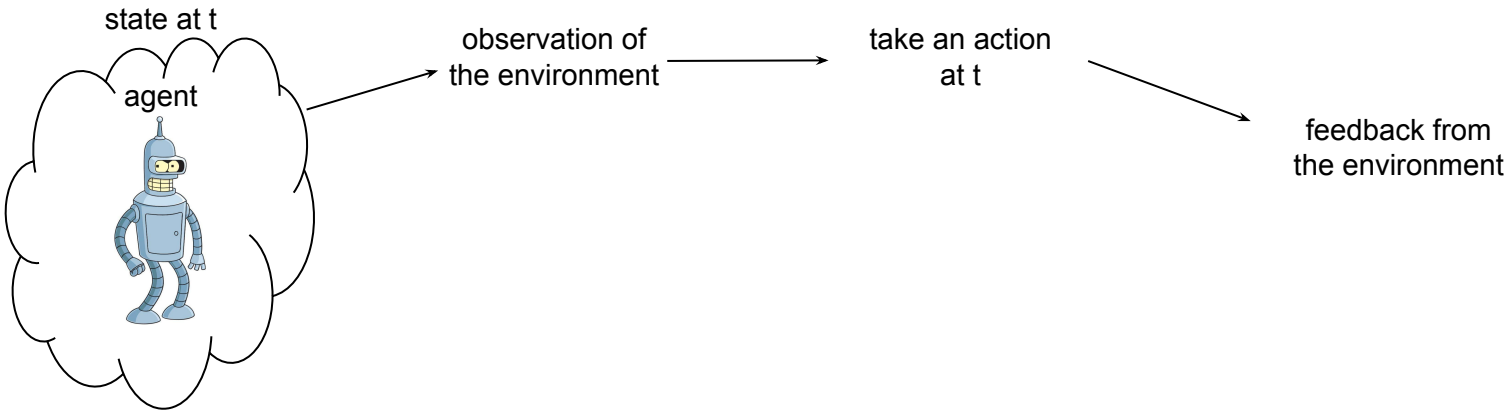


The **Agent** is the entity that aims to solve the task

The **State** is the situation in which the Agent finds itself at time t: it contains all the information about what the Agent knows

The **Environment** is everything that surrounds the agent. Based on agent's **Actions**, it provides **Rewards**

Learning happens in a feedback loop:



# Project introduction: RL basics

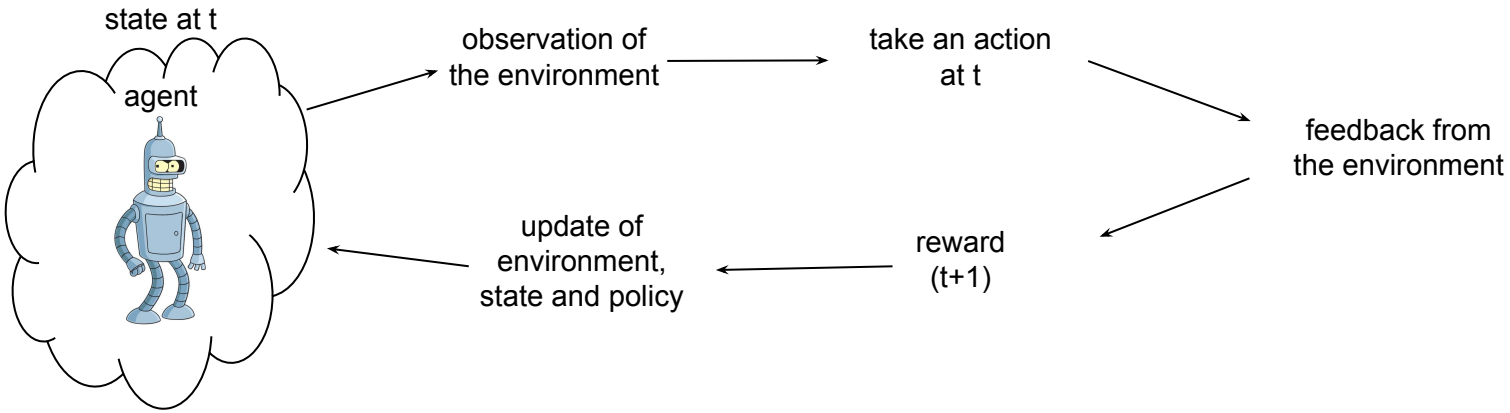


The **Agent** is the entity that aims to solve the task

The **State** is the situation in which the Agent finds itself at time t: it contains all the information about what the Agent knows

The **Environment** is everything that surrounds the agent. Based on agent's **Actions**, it provides **Rewards**

Learning happens in a feedback loop:



# Project introduction: RL basics

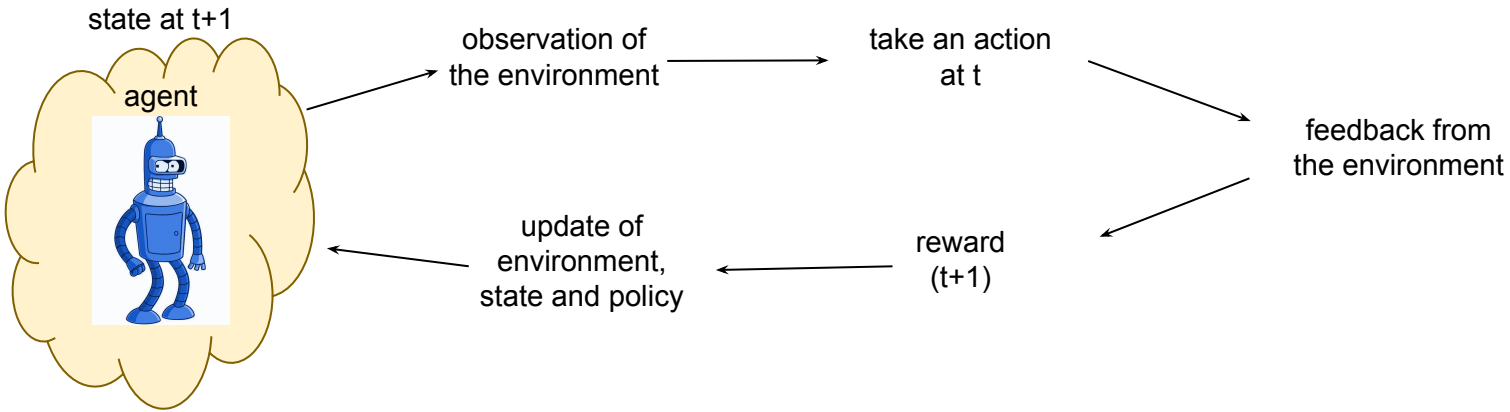


The **Agent** is the entity that aims to solve the task

The **State** is the situation in which the Agent finds itself at time t: it contains all the information about what the Agent knows

The **Environment** is everything that surrounds the agent. Based on agent's **Actions**, it provides **Rewards**

Learning happens in a feedback loop:



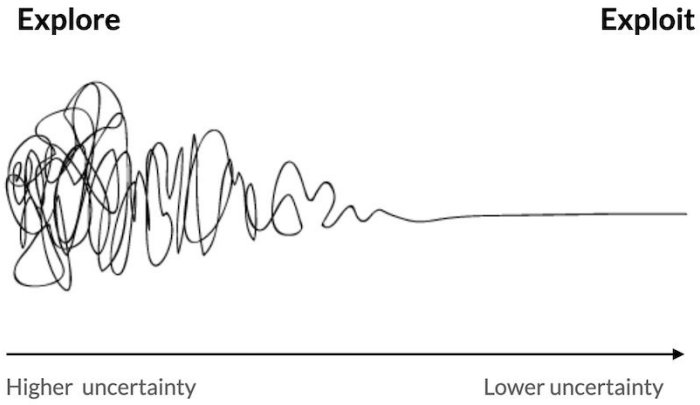
# Project introduction: RL basics



In RL there are different ways of improving the **Policy** (that is, how the agent chooses action based on the state):

- estimate an action-value function Q(s,a) for all pairs and then maximize it
- directly improve the policy

Usually when a choice of policy maximizes the reward we say it is **greedy** towards it. Though we must ensure that our policy is the absolute best: we need to face the **exploitation - exploration tradeoff**



# Project introduction: RL basics



In RL there are different ways of improving the **Policy** (that is, how the agent chooses action based on the state):

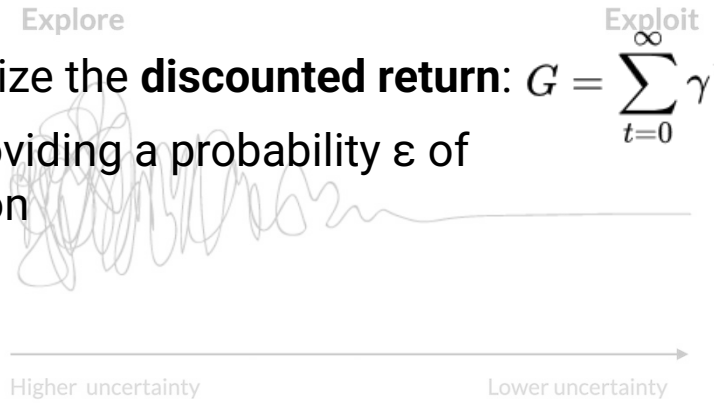
- estimate an action-value function Q(s,a) for all pairs and then maximize it
- directly improve the policy

Usually when a choice of policy maximizes the reward we say it is **greedy** towards it. Though we must ensure that our policy is the absolute best: we need to face the **exploitation - exploration tradeoff**

RL approaches aim to maximize the **discounted return**: $G = \sum_{t=0}^{\infty} \gamma^t R_{t+1} = R_1 + \gamma R_2 + \gamma^2 R_3 + \dots$

$\gamma < 1$

and ensure exploration by providing a probability ε of not choosing the greedy action



# Project introduction: RL basics



Traditional RL approaches aim to solve the problem in the **best possible way** taking the **least possible time**

# Project introduction: RL basics



Traditional RL approaches aim to solve the problem in the **best possible way** taking the **least possible time**

They don't care about **accurate estimates**, nor complete definition of the problem landscape

# Project introduction: RL basics



Traditional RL approaches aim to solve the problem in the **best possible way** taking the **least possible time**

They don't care about **accurate estimates**, nor complete definition of the problem landscape

Bayesian Inference can be used to both **increase the performance** (in terms of episodes) and provide **additional information** about the problem landscape

# Project introduction: RL basics



Traditional RL approaches aim to solve the problem in the **best possible way** taking the **least possible time**

They don't care about **accurate estimates**, nor complete definition of the problem landscape

Bayesian Inference can be used to both **increase the performance** (in terms of episodes) and provide **additional information** about the problem landscape

# Project introduction: goals



Traditional RL approaches aim to solve the problem in the **best possible way** taking the **least possible time**

They don't care about **accurate estimates**, nor complete definition of the problem landscape

Bayesian Inference can be used to both **increase the performance** (in terms of episodes) and provide **additional information** about the problem landscape

Our **goals** are then:

- set up Bayesian Inference algorithms and apply them on Reinforcement Learning problems
- estimate the performance of these algorithms and compare them with traditional RL approaches
- understand the type of RL problems Bayesian approaches are most suitable for

# Application 1: K-armed Bandits

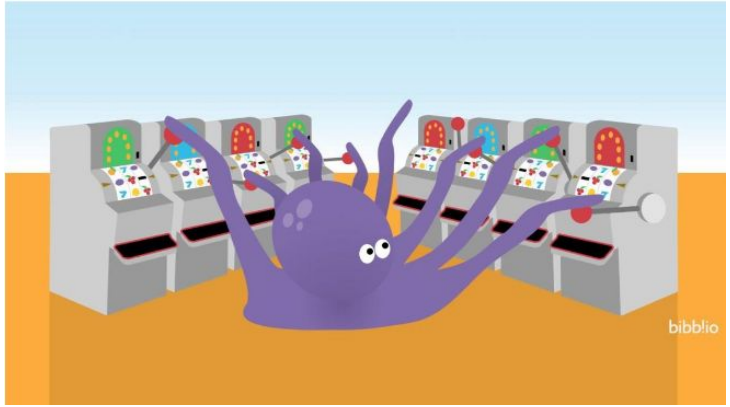
# Problem introduction



**K-armed Bandit** is one of the simplest problems in RL: the agent must **choose a lever out of k** and decide which one **provides the highest** amount of **reward** when pushed.

It is simple because **the state does not change**: every time a lever is pushed the environment "resets", so we just need to figure out what is the best action (lever choice) overall.

Levers can be seen as slot machines, since in general the **reward distribution** can be **stochastic** and non trivial.

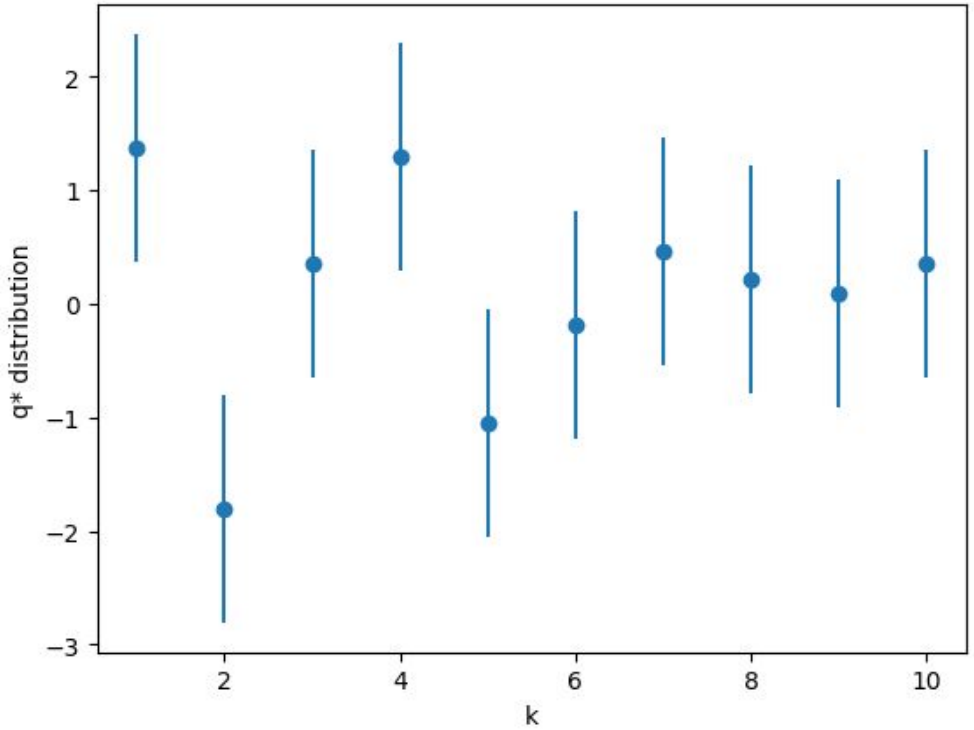


# Problem introduction



In our first setting of the problem, we have K=10 levers, and the **distribution of rewards is gaussian,** with a **fixed known variance $\sigma^2$=1**.

The mean of each gaussian **q*** is a value **extracted from a standard normal** ($\mu$=0 and $\sigma^2$=1).



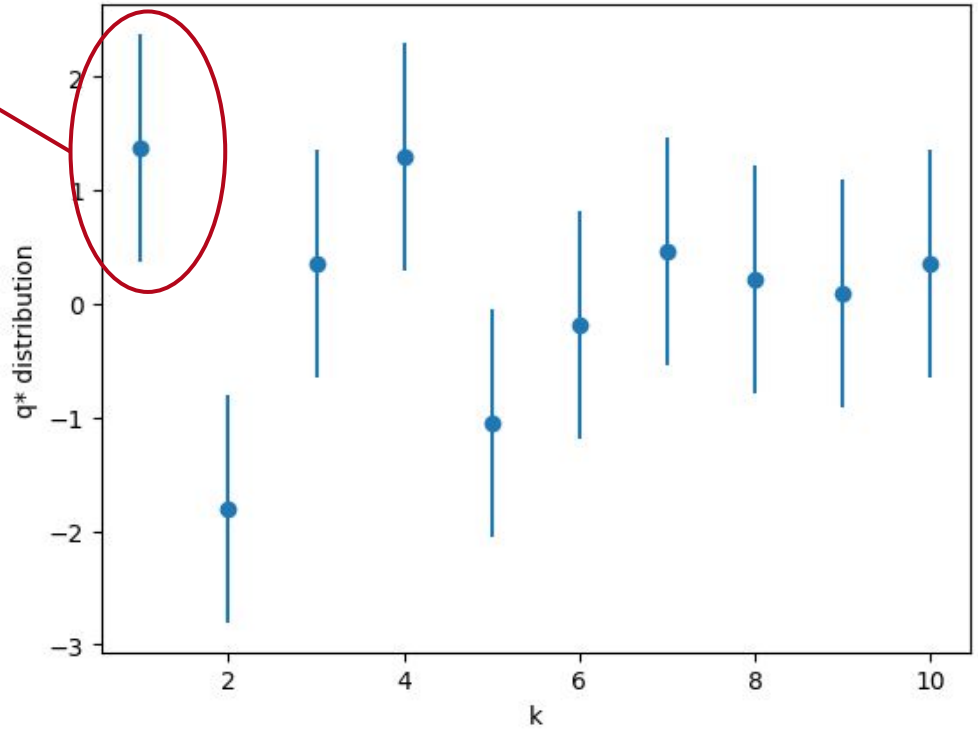
# Problem introduction



we would like our agent to always press lever 1, but it has to figure it out that it is the best!

In our first setting of the problem, we have K=10 levers, and the **distribution of rewards is gaussian,** with a **fixed known variance $\sigma^2=1$**.

The mean of each gaussian **q*** is a value **extracted from a standard normal** ($\mu=0$ and $\sigma^2=1$).



# k-Bandits: Classic RL algorithms

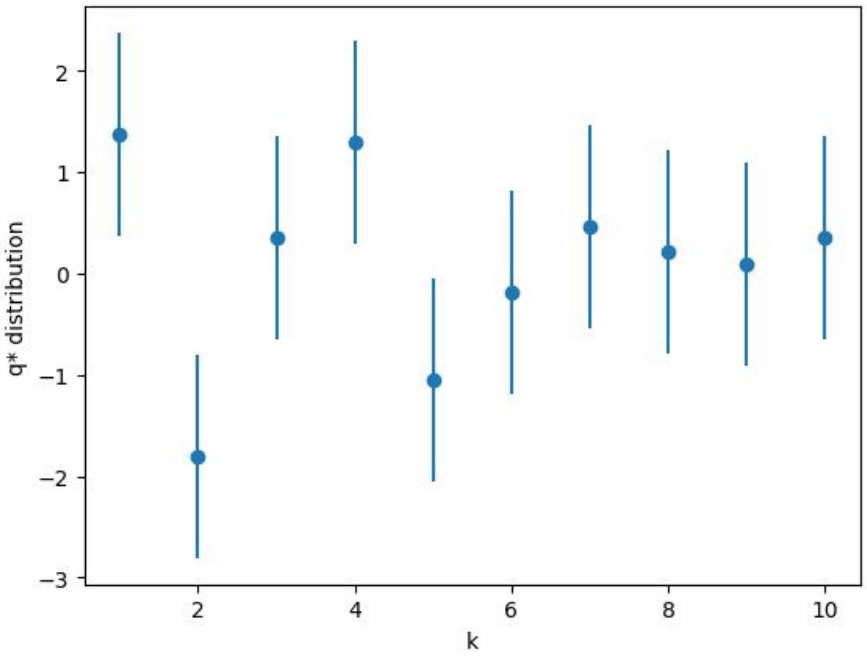


We need the agent to be able to estimate how good each action A (arm choice) can be. To do this we use the **estimated action value Q:**

$$Q_t(A) = \frac{\sum R_i}{N}$$

corresponding to the mean of the rewards obtained from picking action A for N times.

**Q converges to q* of the arm for a large amount of trials**. This enables the agent, with sufficient trial and error, to be able to identify correctly which action is the best one to take, identified as the **greedy action**.



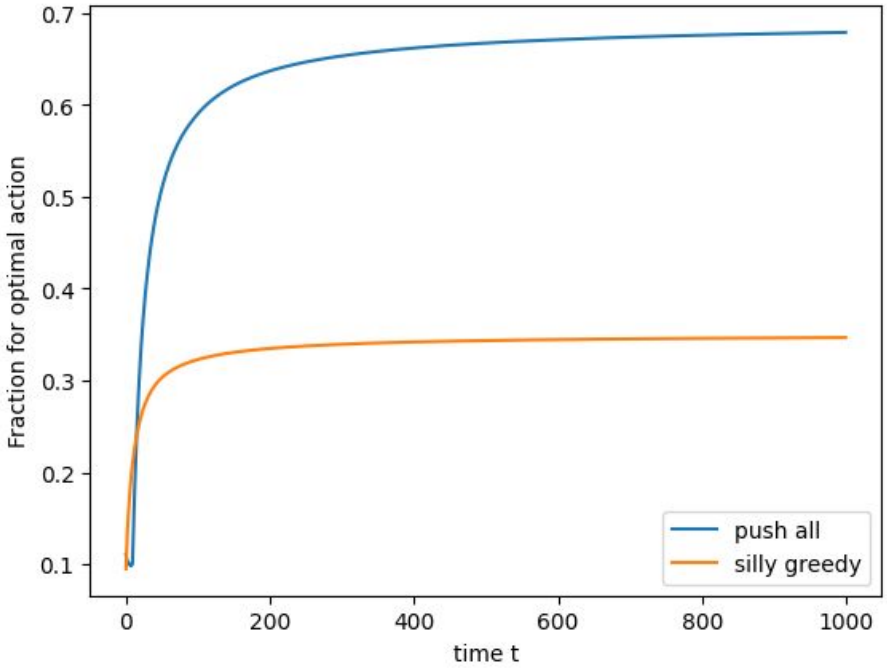
# Classic RL algorithms: Greedy choice



We would like the agent to be able to always pick the greedy action to get the highest amount of rewards.

- One (naive) algorithm is to try once all k levers and then stick to the action found to give the highest reward amount in those trials: **push-all greedy** choice.

- We can also just pick the highest Q-value lever (without trying all k levers before), breaking ties randomly picking an arm: **silly greedy** algorithm.

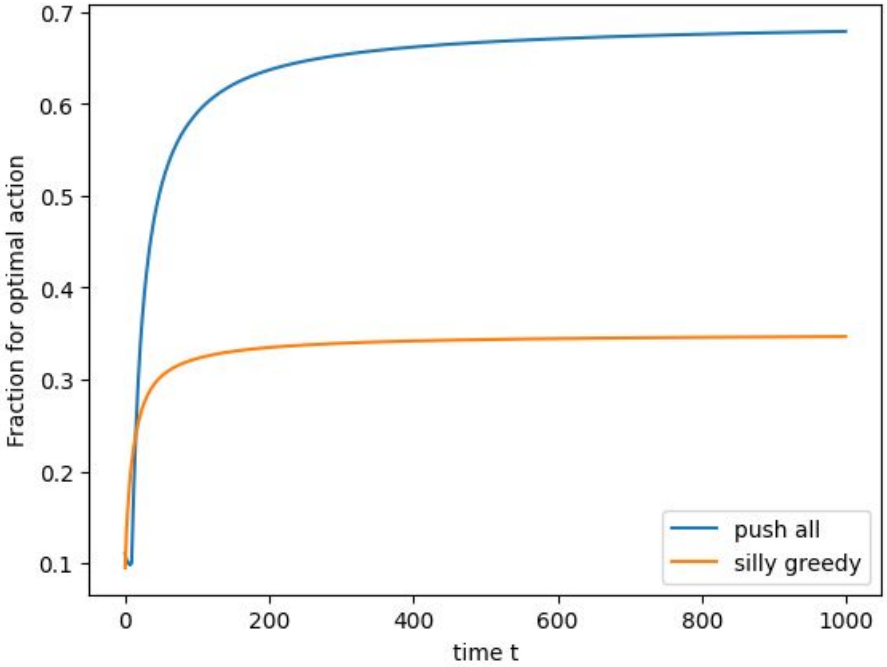


# Classic RL algorithms: greedy choice



- Just pressing once each lever allows the agent to have a significantly **better performance** over time**, thanks to the exploration** performed at the start.

- The agent sacrifices some rewards initially to maximize long term ones. **We would like to optimize this exploration-exploitation tradeoff** even more, by identifying as soon as possible the best lever.

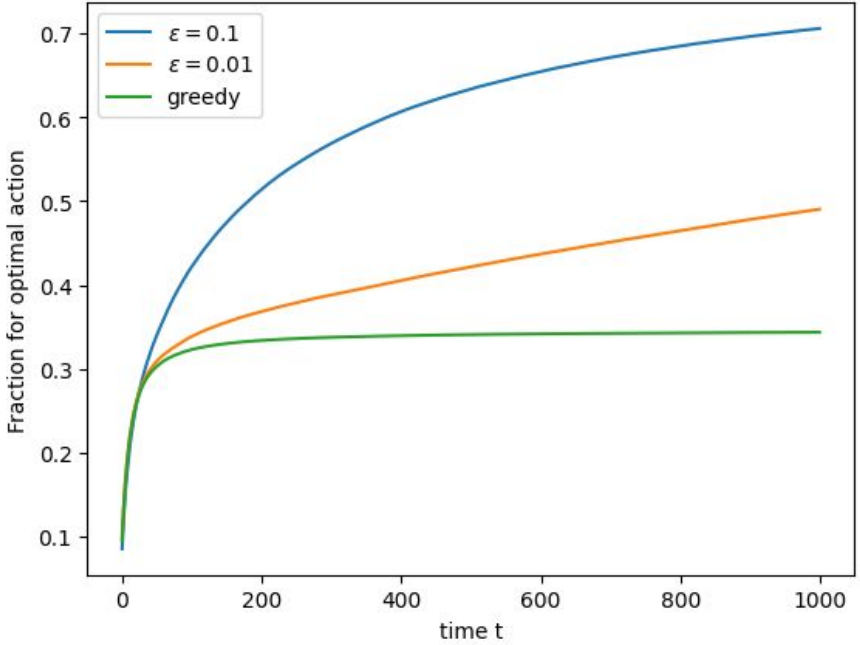


# Classic RL algorithms: ε-greedy choice



- One possibility is **ε-greedy choice**: define a small quantity ε that describes the importance of the exploration for the agent. It will take with probability 1-ε the current greedy action, while with probability ε it will take another action at random.

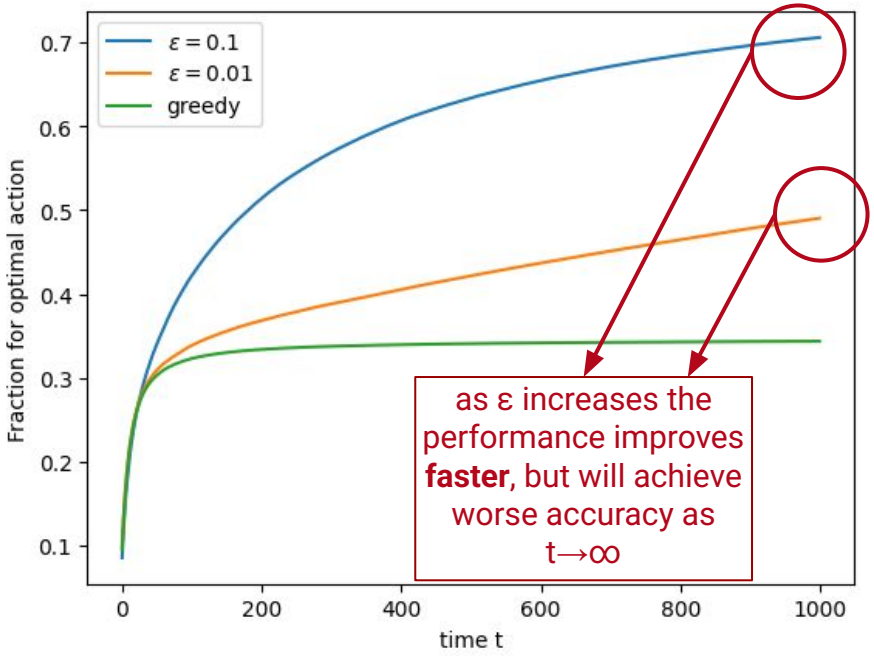
- This leads the agent to not stick to a presumed “best action”, but to try out others and look for improvements.



# Classic RL algorithms: ε-greedy choice



- As ε increases, the accuracy increases faster, but it will approach 1-ε good choices at the end. The algorithm will still fail a fraction ε of times at convergence
- To achieve better performance, the value of ε could be adjusted (reduced) according to the timestep t.



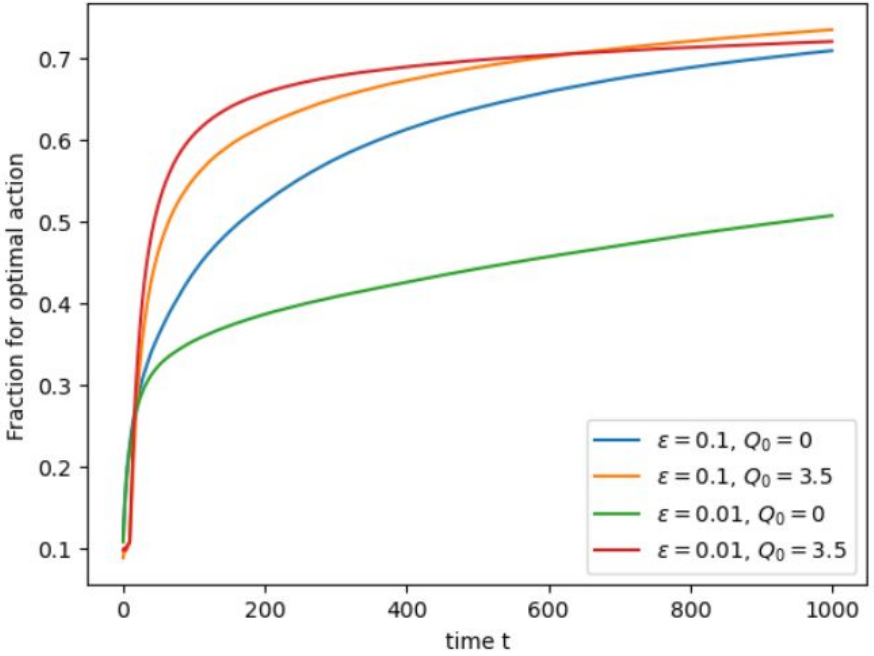
# Classic RL algorithms: Optimistic Initial Values



- By providing **"optimistic" initial $Q_0$ for the actions values**, the agent finds itself thinking some actions will yield even higher rewards than they could ever do. This **forces the agent to explore more at the beginning**, as the Q-values converge to the true mean.

- To apply the algorithm **we need some kind of prior knowledge about the problem**. In this context, optimistic initial values are for example in $Q_0$>3 range.

- A combination of both **ε-greedy** and **OIV** can be implemented.



# Classic RL algorithms: Upper Confidence Bound



We would like to do better than picking the non-greedy action with uniform probability.This can be achieved considering a **function that accounts for the uncertainty associated with each lever** (how confident we are for each single action), together with the expected reward. The new action choice becomes:

$$A(t) = argmax(Q_i(t) + c\sqrt{ln(t)/N_i(t)})$$

# Classic RL algorithms: Upper Confidence Bound



We would like to do better than picking the non-greedy action with uniform probability. This can be achieved considering a **function that accounts for the uncertainty associated with each lever** (how confident we are for each single action), together with the expected reward. The new action choice becomes:

$$A(t) = argmax(Q_i(t) + c\sqrt{ln(t)/N_i(t)})$$

standard bandit estimate

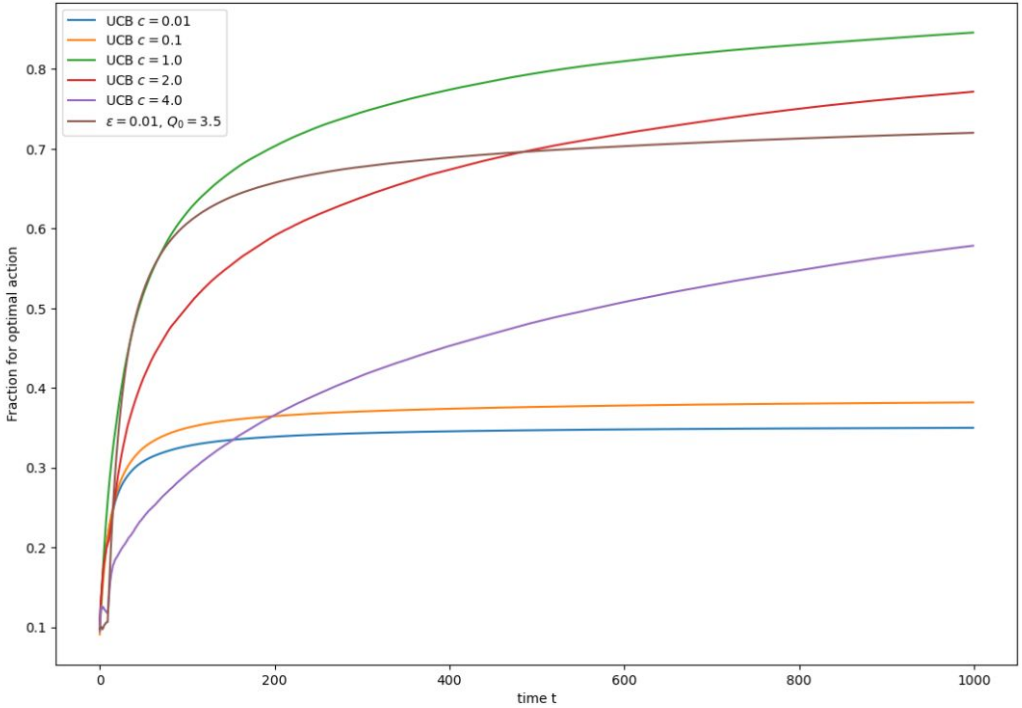
uncertainty estimate

- N is an array containing how many times each specific arm has been pulled so-far.
- c is a hyperparameter (exploration degree) that controls how much we value uncertainty.

If an arm hasn't been pulled much, $N_i$ is small, making the exploration bonus very large and forcing the algorithm to pick that arm, even if its current $Q_i$ is low.

# Classic RL algorithms: Upper Confidence Bound






- **Upper Confidence Bound** with **c=1** clearly **obtains the best result among the classical RL algorithms considered** so far (the fastest to learn the best action and to exploit it).
- However, we observe a very **strong variability with different c** values.

# K-Bandits: Bayesian implementation



Now the agent maintains a **posterior probability distribution** (belief) **for the mean reward of each arm**.

To do this we use **Thompson sampling,** consisting in:
- **Sampling**: at every step, the agent draws one random sample from each arm's posterior distribution of the mean (starting from a certain initial prior).
- **Action**: it picks the arm that produced the highest sample among these, and a reward is obtained.
- **Update**: after observing the reward, we use Bayes' Theorem to update the posterior of each arm.

We consider **two settings**: a first simpler case in which we have an **analytical posterior** to sample (corresponding to the same previous problem, with a gaussian for the prior), and a **more general** one in which we do **not** have an **analytical posterior.**

# Bayesian K-Bandit: analytical posterior



- **Same setting as before**: we have 10 levers, and the "true" mean reward of each arm is drawn at each game from a standard normal: $q^* \sim \mathcal{N}(0,1)$

- When pulled, the reward of each lever *a* is random, distributed as a gaussian with a **fixed known variance**: $R \sim \mathcal{N}(q_a^*, 1.0)$

- We initialize the agent's beliefs as a **gaussian** distribution with $\mu_0$=0 and $\sigma_0$=100, indicating a **highly uninformative (flat) prior**.

The prior and the posterior are **conjugate distributions**

- In particular the **posterior is a Gaussian** with

$$\mu_{post} = \sigma^2_{post}\left(\frac{\mu_{prior}}{\sigma^2_{prior}} + \frac{R}{\sigma^2_{like}}\right) \qquad \frac{1}{\sigma^2_{post}} = \frac{1}{\sigma^2_{prior}} + \frac{1}{\sigma^2_{like}}$$

Known! (fixed to 1)

# Bayesian K-Bandit: analytical posterior



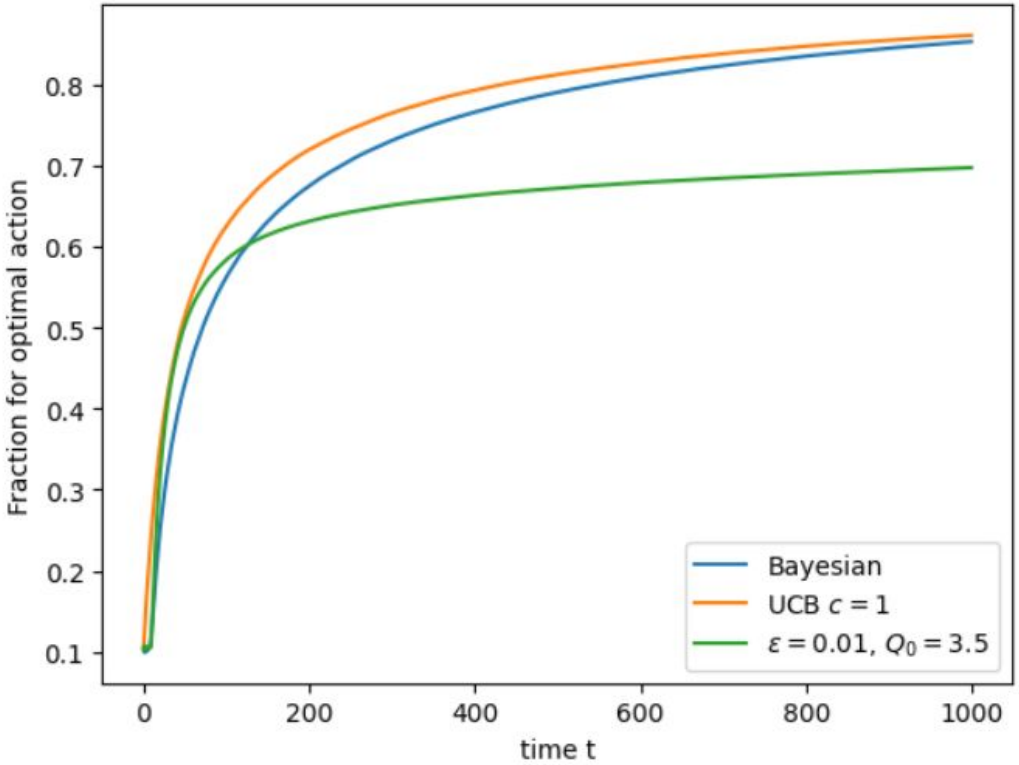
- At each update of the Thompson sampling **we are able to sample the gaussian posterior** for the mean of each arm, we then choose the arm with the highest sampled value.

- A useful **metric to compare** performance of the different methods (classical RL vs Bayesian-based) is **the average regret across multiple runs,** defined as follows:
  - For each run, consider the curve of the **fraction of optimal actions** vs timestep
  - regret is **1 - AUC** of this curve, where the area under the curve is normalized to the number of timesteps.

- In this way we consider which algorithm has both **faster convergence and higher accuracy.**

# Results comparison






| Algorithm | Regret |
|---|---|
| Push-all greedy | 0.353 ± 0.009 |
| Silly greedy | 0.661 ± 0.01 |
| ε-greedy (ε=0.1) | 0.403 ± 0.007 |
| OIV (ε=0.1, $Q_0$=3.5) | 0.393 ± 0.007 |
| **UCB (c=1)** | **0.242 ± 0.005** |
| **Bayesian** | **0.273 ± 0.004** |

# Results comparison



- For this simple setting, we observe that the analytical **Bayesian approach** (assuming an almost flat prior) actually **sligthly underperfoms the best UCB** classical RL algorithm, which also defines a quantitative notion of the uncertainty of each arm.

- However, **Bayesian approach has the** important **advantage that it does not require to "manually" fine-tune the algorithm's hyper-parameters** (UCB is very sensitive to c) to get the most efficient performance, while still obtaining a low regret. In a real case scenario in fact one cannot actually evaluate the regret, we can just play the game again with a different algorithm and check the average reward.

| Algorithm | Regret |
|---|---|
| Push-all greedy | 0.353 ± 0.009 |
| Silly greedy | 0.661 ± 0.01 |
| ε-greedy (ε=0.1) | 0.403 ± 0.007 |
| OIV (ε=0.1, $Q_0$=3.5) | 0.393 ± 0.007 |
| **UCB (c=1)** | **0.242 ± 0.005** |
| **Bayesian** | **0.273 ± 0.004** |

# Bayesian K-Bandit: general case



- We consider a **more general case**: when pulled, the reward of each lever *a* is random, distributed as a gaussian with a **certain mean and variance, both unknown to the agent**:

$$R \sim \mathcal{N}(q_a^*, \sigma_a^{*2})$$

- We initialize the agent's **prior belief for the mean** of each arm **with a T-student** distribution, in order to allow the possibility of large tails for the rewards distributions. For the **prior of $\sigma_a$*** we use an **half-normal distribution** (positive).

⇩

**We do not have analytical posteriors to sample**

- In order to apply Thompson sampling, we consider two possible methods that approximate and sample from the posterior distribution at each update: **MCMC-based and Variational inference**.

# Bayesian K-Bandit: MCMC



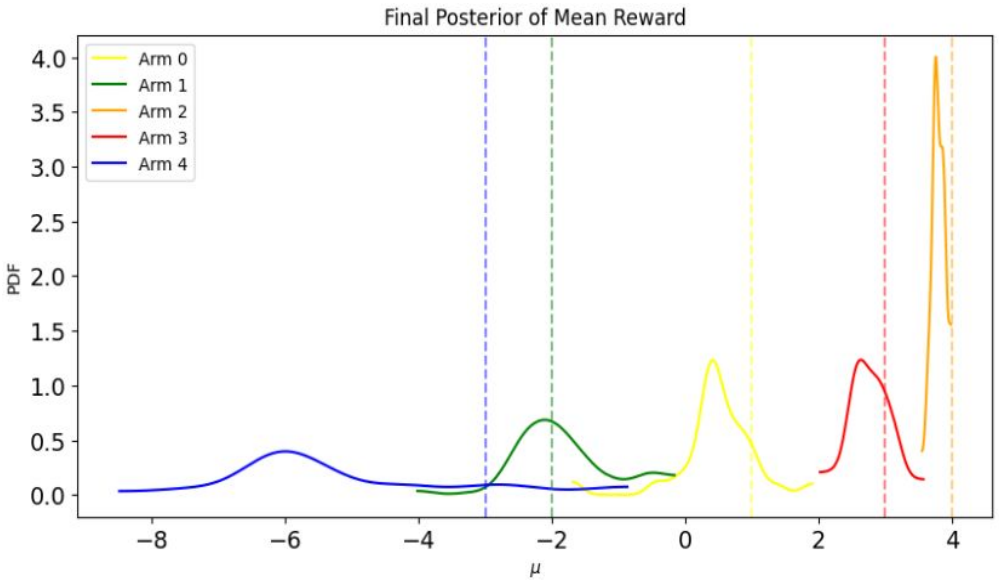
- At each update:
  - we use **MCMC to** numerically approximate the posteriors and **obtain samples of the mean and variance** from it at convergence
  - we then use again Thompson sampling, randomly picking a single sample for the means, and choosing as action the arm with the highest mean in this sample.
  - we obtain a reward, and we can **update the posterior** distribution **to approximate by considering a cumulative likelihood based on the history of all rewards** (product of likelihoods).

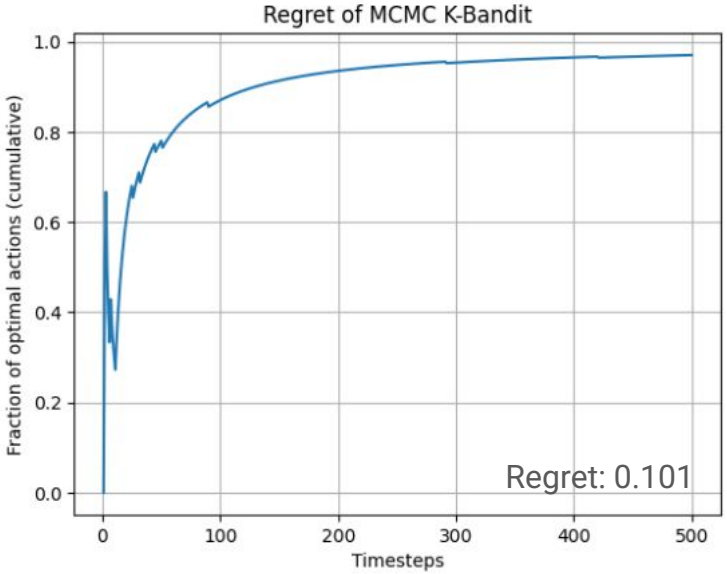
- We use **NUTS sampler**: it runs (multiple parallel) chain(s) applying **Hamiltonian Monte Carlo**.

# Bayesian K-Bandit: MCMC



Here we simulate a **single run with 5 arms** (computational reasons), played for 500 iterations. At each iteration we draw 100 samples from the MC when at convergence (burn in of 1000) and then randomly pick one.

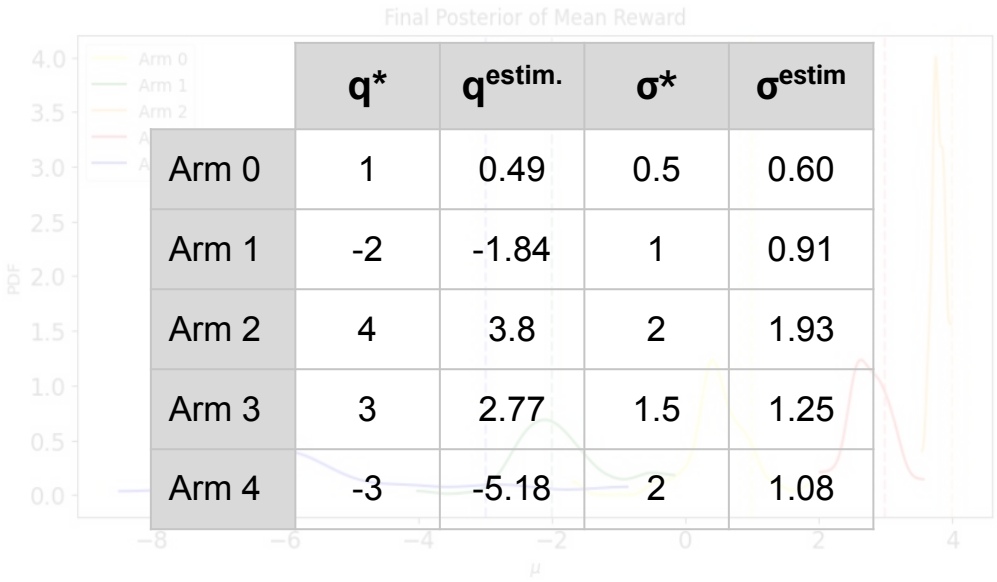


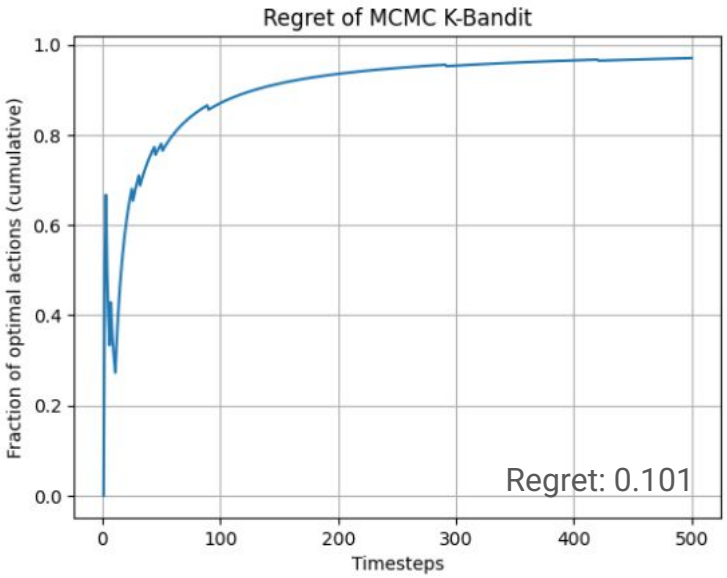
# Bayesian K-Bandit: MCMC



Here we simulate a **single run with 5 arms** (computational reasons), played for 500 iterations. At each iteration we draw 100 samples from the MC when at convergence (burn in of 1000) and then randomly pick one.



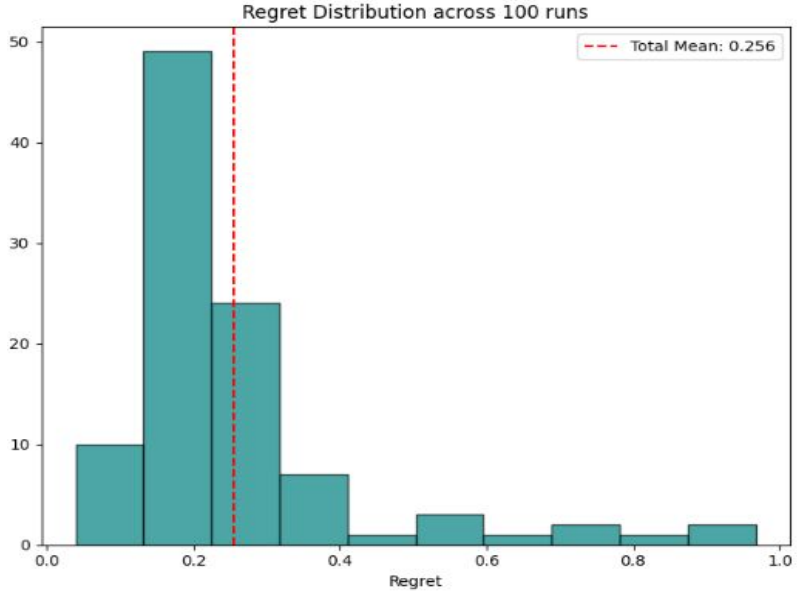

| | $q^*$ | $q^{estim.}$ | $\sigma^*$ | $\sigma^{estim}$ |
|---|---|---|---|---|
| Arm 0 | 1 | 0.49 | 0.5 | 0.60 |
| Arm 1 | -2 | -1.84 | 1 | 0.91 |
| Arm 2 | 4 | 3.8 | 2 | 1.93 |
| Arm 3 | 3 | 2.77 | 1.5 | 1.25 |
| Arm 4 | -3 | -5.18 | 2 | 1.08 |

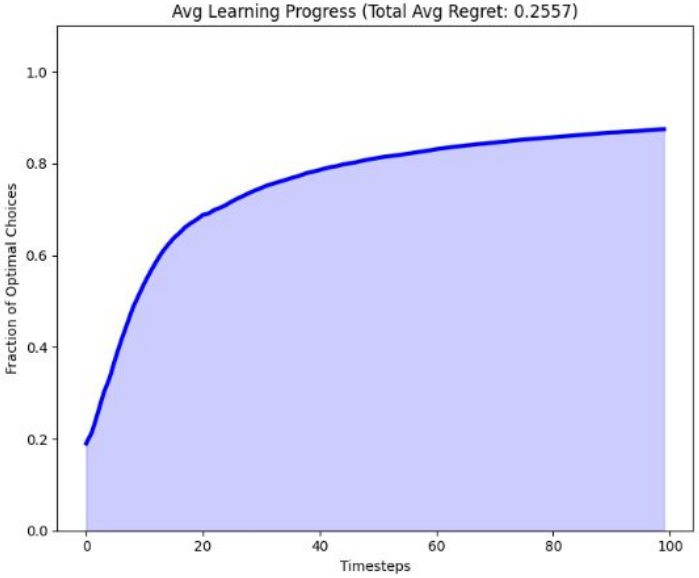


# Bayesian K-Bandit: MCMC



We then **run 100 different simulations of the game** on a CloudVeneto GPU, each with 100 timesteps, to get an estimate the mean regret of the algorithm: 0.256±0.017




- Very high variability of regrets! (at least in the first 100 steps)
- Computationally very heavy (slow)

# Bayesian K-Bandit: Variational Inference



- The goal of this method is to infer the posterior p(θ | D) by approximating it with a simpler distribution q(θ).
- q(θ) is chosen by minimizing the KL-Divergence, KL(q // p).
- To do so we actually maximize the ELBO (Evidence Lower BOund).

$$ELBO = E_q[ln(p(D,\theta))] - E_q[ln(q(D,\theta))]$$

The problem is set up the same way as for the cases before.

# Bayesian K-Bandit: Variational Inference



The reward is modelled as a mixture of K-gaussians (K=2 in our case)[3]

$$R(y) = \sum_{k=1}^{K} \pi_{a,k} \, \mathcal{N}(\mu_{a,k}, \sigma^2_{a,k})$$

For each of the parameters we assume a prior that gets updated at each cycle.

- Dirichlet for the weights of each gaussian component.

$$p(\pi_a) = \mathrm{Dir}(\pi_a \mid \gamma_0)$$

- Normal-Inverse-Gamma for mean and variance of the gaussians

$$p(w_{a,k},\, \sigma^2_{a,k}) = \mathrm{NIG}(w_{a,k},\, \sigma^2_{a,k} \mid u_0,\, V_0,\, \alpha_0,\, \beta_0)$$

# Bayesian K-Bandit: Variational Inference



**Updata Algorithm:**

For each arm *a* and each mixture component *k*, sample from the parameters distributions:

# Bayesian K-Bandit: Variational Inference



**Updata Algorithm:**

For each arm *a* and each mixture component *k*, sample from the parameters distributions:

$$\tilde{\pi}_a \sim Dir(\tilde{\gamma}_a)$$
$$\tilde{\sigma}^2_{a,k} \sim \Gamma^{-1}(\tilde{\alpha}_{a,k}, \tilde{\beta}_{a,k})$$
$$\tilde{\mu}_a \sim \mathcal{N}(\tilde{u}_{ak}, \tilde{\sigma}^2_{a,k}\tilde{V}_{a,k})$$

Calculate the reward $\longrightarrow$

$$\hat{\mu}_a = \sum_{k=1}^{K} \tilde{\pi}_{a,k}\tilde{\mu}_{a,k}$$

# Bayesian K-Bandit: Variational Inference



**Updata Algorithm:**

For each arm *a* and each mixture component *k*, sample from the parameters distributions:

$$\tilde{\pi}_a \sim Dir(\tilde{\gamma}_a)$$
$$\tilde{\sigma}^2_{a,k} \sim \Gamma^{-1}(\tilde{\alpha}_{a,k}, \tilde{\beta}_{a,k})$$
$$\tilde{\mu}_a \sim \mathcal{N}(\tilde{u}_{ak}, \tilde{\sigma}^2_{a,k}\tilde{V}_{a,k})$$

$$\xrightarrow{\text{Calculate the reward}} \quad \hat{\mu}_a = \sum_{k=1}^{K} \tilde{\pi}_{a,k}\tilde{\mu}_{a,k}$$

**Thompson Sampling** to select the best arm and the reward

$$a^*_{t+1} = argmax_a(\hat{\mu}_a) \qquad y_{t+1} \sim \mathcal{N}(q^*_{a^*}, \sigma_{a^*})$$

# Bayesian K-Bandit: Variational Inference



**Update Posteriors:**

Each parameter of the distributions is updated based on the chosen arm (formulas from *I.Urteaga,C.H.Wiggins* paper.[3])

$$\tilde{\gamma}_{a,k} = \gamma_0 + R_{a,k}$$
$$\tilde{V}_{a,k} = \left(V_0^{-1} + R_{a,k}\right)^{-1}$$
$$\tilde{u}_{a,k} = \tilde{V}_{a,k}\left(\frac{u_0}{V_0} + \sum_t r_{a,k,t}\, y_t\right)$$
$$\tilde{\alpha}_{a,k} = \alpha_0 + \frac{R_{a,k}}{2}$$
$$\tilde{\beta}_{a,k} = \beta_0 + \frac{1}{2}\sum_t r_{a,k,t}(y_t - \tilde{u}_{a,k})^2 + \frac{(\tilde{u}_{a,k} - u_0)^2}{2V_0}$$

Where *r* are the **responsibilities**, the posterior probability that a data point belongs to a specific component *k*.

$$r_{ik} = \frac{\rho_{ik}}{\sum_k \rho_{ik}} \qquad \text{with} \qquad \rho_{ik} = e^{E_{\mathbf{p}}[\log p_k]} e^{\frac{1}{2} E_{\boldsymbol{\mu}}[(x_i - \mu_k)^2]/\sigma^2}$$

$$R_{a,k} = \sum_t r_{a,k,t}$$

# Bayesian K-Bandit: Variational Inference



We use ELBO as a **convergence criteria**.

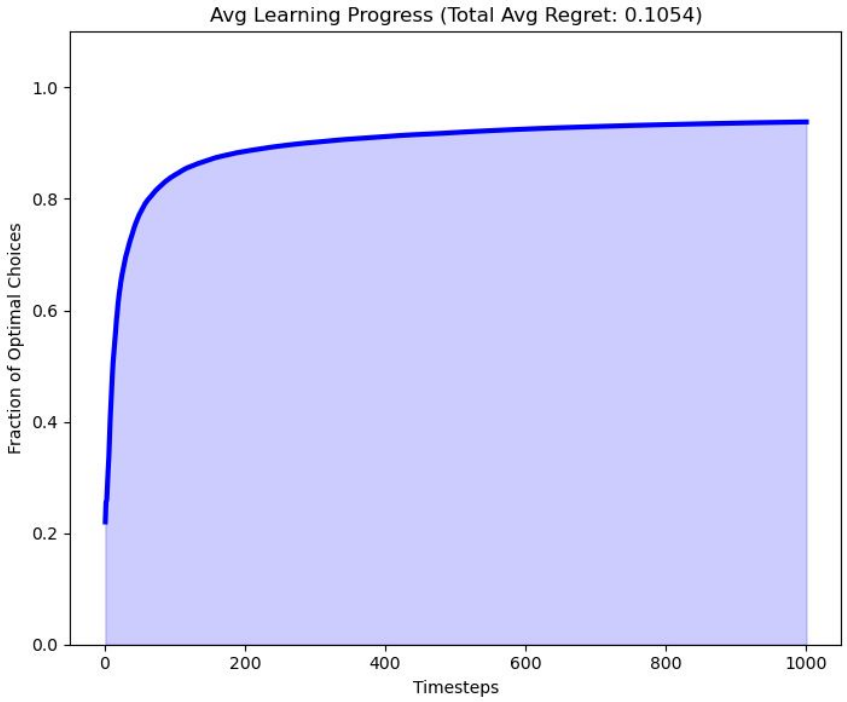
ELBO is a lower bound for the log-likelihood with 3 terms: [4]

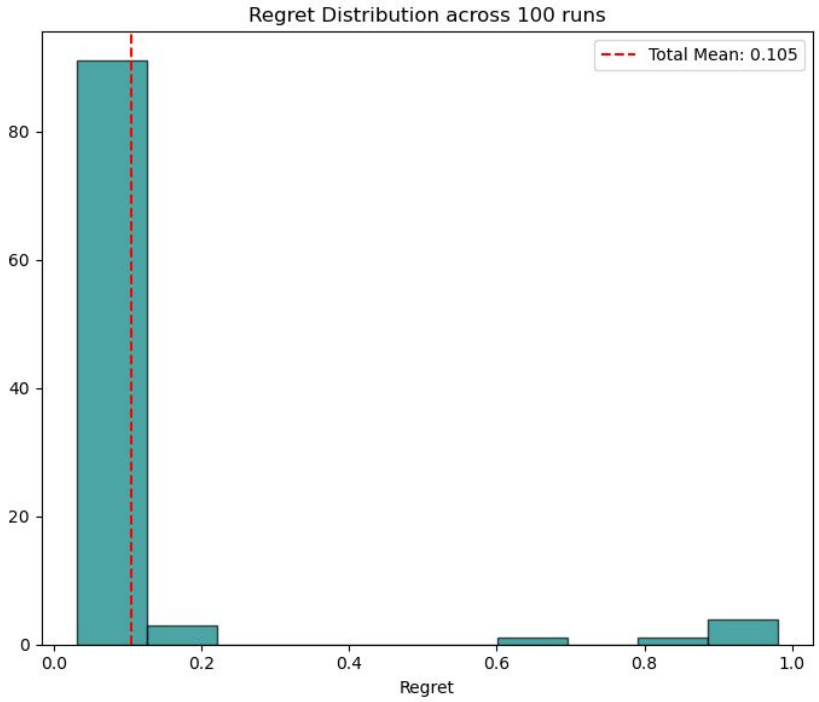
- Expected log-likelihood: how well the model fits the observed rewards
- Entropy: encourage exploration
- KL from priors: penalises posteriors that deviate too far from priors

$$ELBO = \underbrace{\mathbb{E}_q[\ln p(\mathcal{D} \mid \theta)]}_{\text{expected log-likelihood}} + \underbrace{\mathbb{E}_q[\ln p(Z \mid \pi)] - \mathbb{E}_q[\ln q(Z)]}_{\text{entropy}} - \underbrace{\sum_a \mathrm{KL}(q(\pi_a) \| p(\pi_a)) + \sum_{a,k} \mathrm{KL}(q(w_{a,k}, \sigma^2_{a,k}) \| p(w_{a,k}, \sigma^2_{a,k}))}_{\text{KL from priors}}$$

# Bayesian K-Bandit: Variational Inference



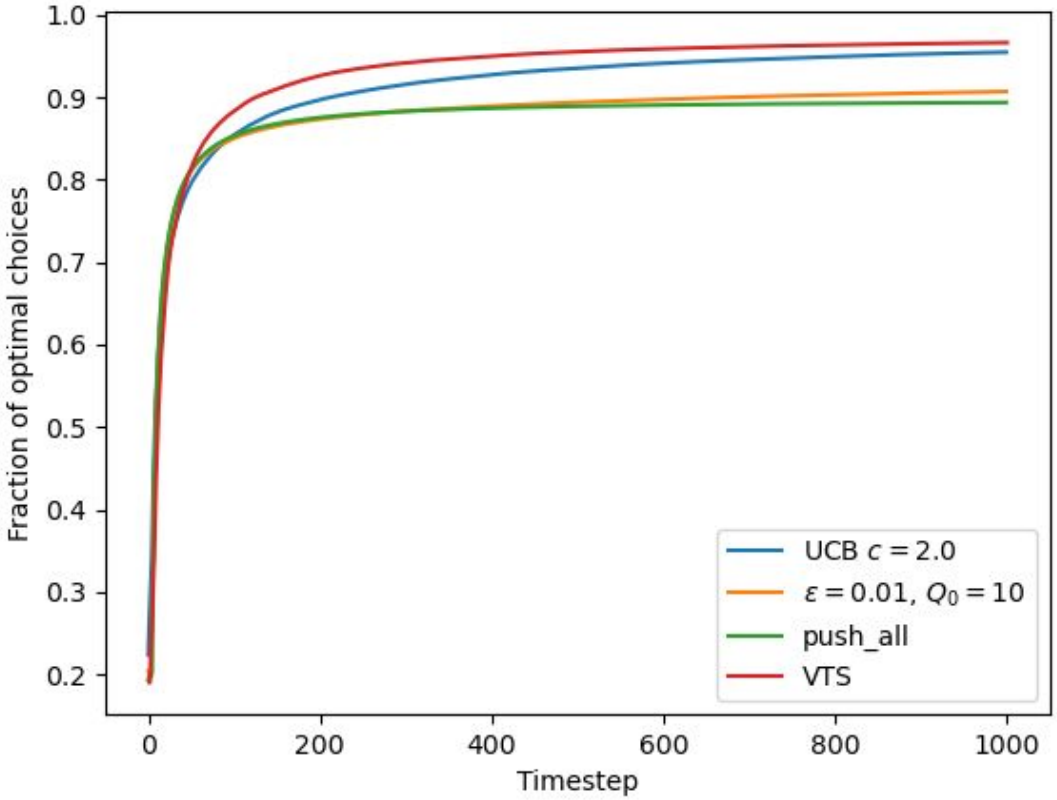



- Good performance
- Low average regret and peaked distribution

# Comparison with classical RL algorithms






| Algorithm | Regret |
|---|---|
| Push-all greedy | 0.128 ± 0.006 |
| OIV (ε=0.1, $Q_0$=10) | 0.167 ± 0.015 |
| **UCB (c=4)** | **0.089 ± 0.013** |
| **VTS** | 0.071 ± 0.011 |
| MCMC | 0.256 ± 0.017 |

# Bayesian K-Bandit: results comparison



Among Bayesian approaches, **Variational Inference** with Thompson Sampling achieves a **better performance than MCMC** for this specific problem, both **in terms of regret and computational time**.

| Algorithm | Regret |
|---|---|
| Push-all greedy | 0.128 ± 0.006 |
| OIV ($\varepsilon$=0.1, $Q_0$=10) | 0.167 ± 0.015 |
| **UCB (c=4)** | **0.089 ± 0.013** |
| **VTS** | 0.071 ± 0.011 |
| MCMC | 0.256 ± 0.017 |

# Bayesian K-Bandit: results comparison



Among Bayesian approaches, **Variational Inference** with Thompson Sampling achieves a **better performance than MCMC** for this specific problem, both **in terms of regret and computational time**.

Despite obtaining comparable performance, unlike classical RL algorithms, **Variational Inference does not require particular hyper-parameters tuning**. As for MCMC methods, it can be generalized to model arbitrary rewards distributions by considering more gaussians in the mixture .

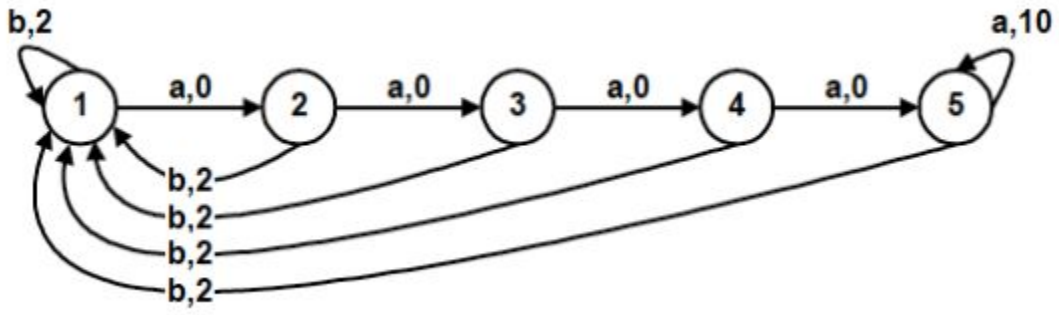
| Algorithm | Regret |
|---|---|
| Push-all greedy | 0.128 ± 0.006 |
| OIV ($\varepsilon$=0.1, $Q_0$=10) | 0.167 ± 0.015 |
| **UCB (c=4)** | **0.089 ± 0.013** |
| **VTS** | 0.071 ± 0.011 |
| MCMC | 0.256 ± 0.017 |

# Application 2: The Chain Problem

# The chain problem: Introduction



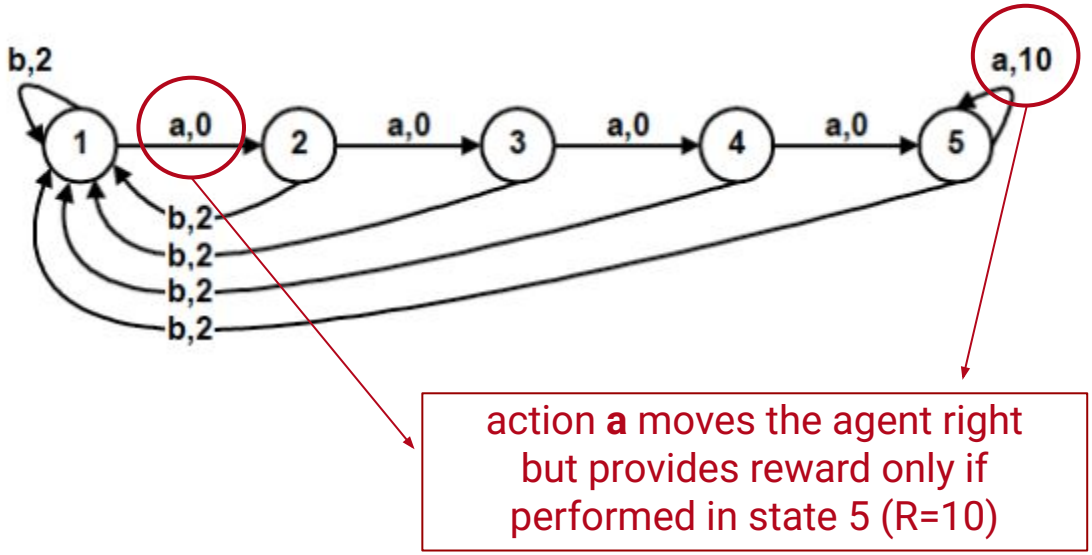
The Chain Problem was introduced by Strens in 2000 [1]. It involves understanding the best strategy in the following chain:



# The chain problem: Introduction



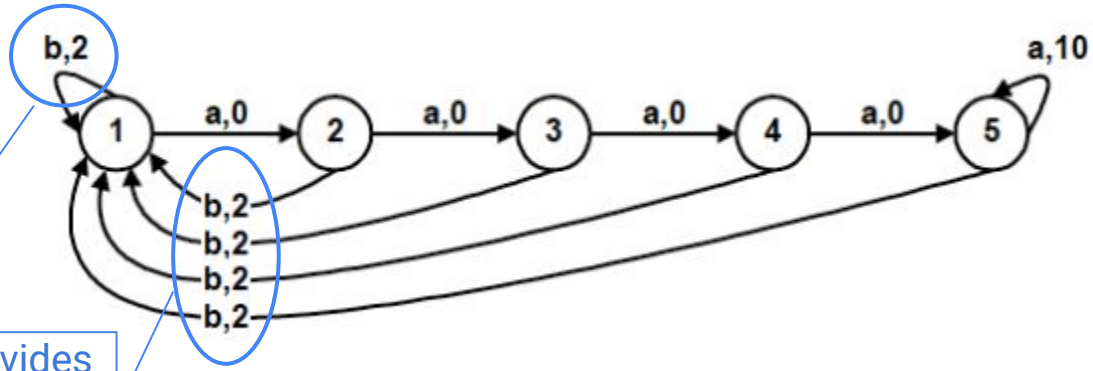
The Chain Problem was introduced by Strens in 2000 [1]. It involves understanding the best strategy in the following chain:



# The chain problem: Introduction



The Chain Problem was introduced by Strens in 2000 [1]. It involves understanding the best strategy in the following chain:

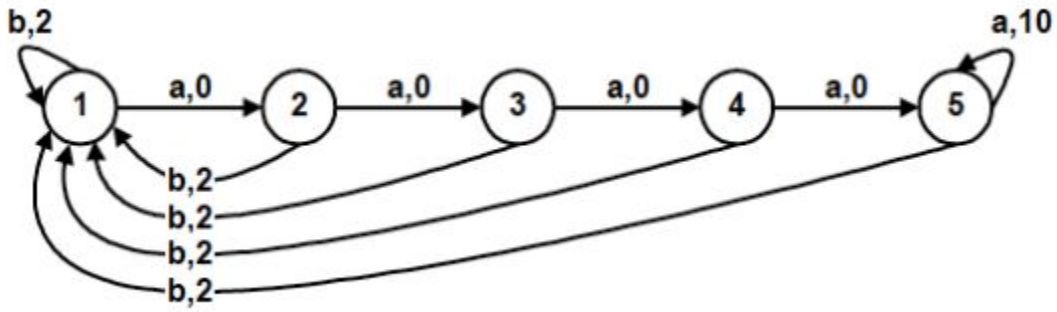



action **b** always provides reward 2 but resets agent to state 1

# The chain problem: Introduction



The Chain Problem was introduced by Strens in 2000 [1]. It involves understanding the best strategy in the following chain:

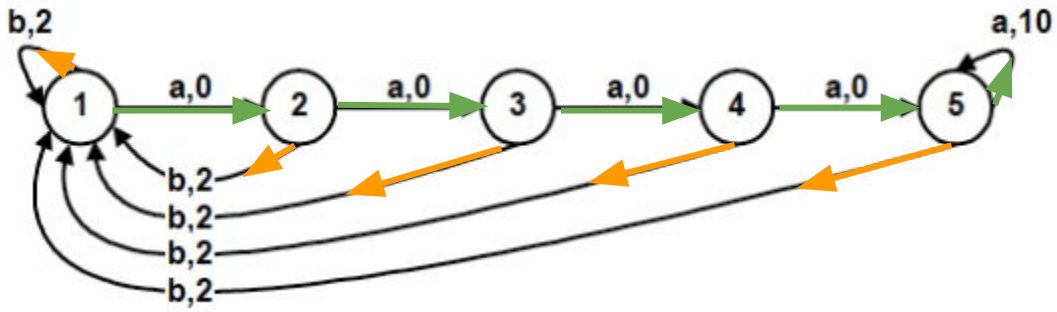



The catch is that every action has a **success rate** of **80%**, leading the agent in unexpected states and with uncertainty relative to its future state

# The chain problem: Introduction



The Chain Problem was introduced by Strens in 2000 [1]. It involves understanding the best strategy in the following chain:




The catch is that every action has a **success rate** of **80%**, leading the agent in unexpected states and with uncertainty relative to its future state

The **best strategy** is try to go right always, to maximize rewards, but algorithms may fall for the **safe strategy**, that is go left always

# The chain problem: Introduction



Since the nature of the problem is **stochastic**, we provide an estimate of the average expected reward per time step

# The chain problem: Introduction



Since the nature of the problem is **stochastic**, we provide an estimate of the average expected reward per time step

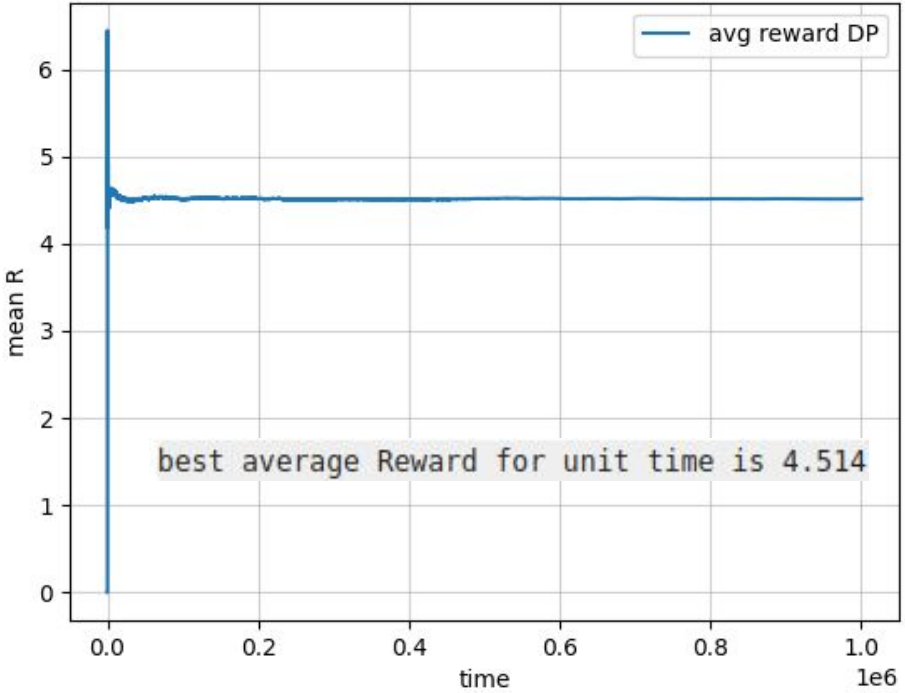
We used **Dynamic Programming** to provide an estimate of the best policy and run a simulation of one million steps to get the average reward per time step

# The chain problem: Introduction



Being the nature of the problem **stochastic**, we provide an estimate of the average expected reward per time step

We used **Dynan** st policy and run a simulation of o step

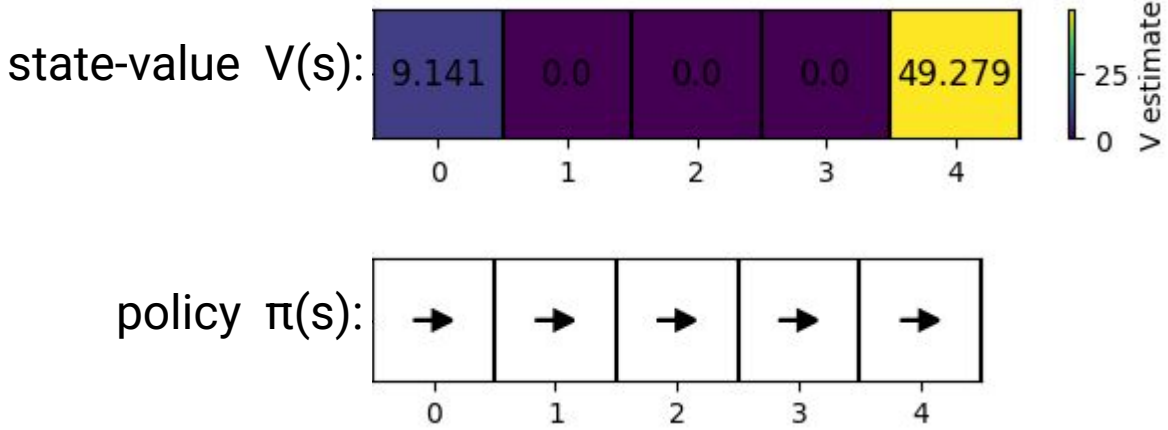


# The chain problem: Introduction



Being the nature of the problem **stochastic**, we provide an estimate of the average expected reward per time step

We used **Dynamic Programming** to provide an estimate of the best policy and run a simulation of one million steps to get the average reward per time step



# The chain problem: MC



We solved The Chain Problem with multiple classic algorithms.
First off with **Monte Carlo**. We show results averaging 100 runs of 1000 episodes




1. Random initialization of policy $\pi$ and state-action values $Q(s, a)$ for all $(s, a)$ pairs
2. Initialization of the episode (initial state) and ensurement that all actions have probability > 0 (ensure exploration)
3. Run a complete episode following $\pi$
4. Compute the returns as $G_t = R_t + G_{t-1}$ for $t = T - 1, \ldots, 0$ (reversing time saves computational resources)
5. (FIRST VISIT), for all $t$, if $(s_t, a_t)$ not in $[(s_{t-1}, a_{t-1}), \ldots, (s_0, a_0))$, append $G$ to returns of $(s_t, a_t)$ and update $Q(s_t, a_t) =< returns(s_t, a_t) >$
6. Go back to 2.

# The chain problem: MC



We solved The Chain Problem with multiple classic algorithms.
First off with **Monte Carlo**. We show results averaging 100 runs of 1000 episodes

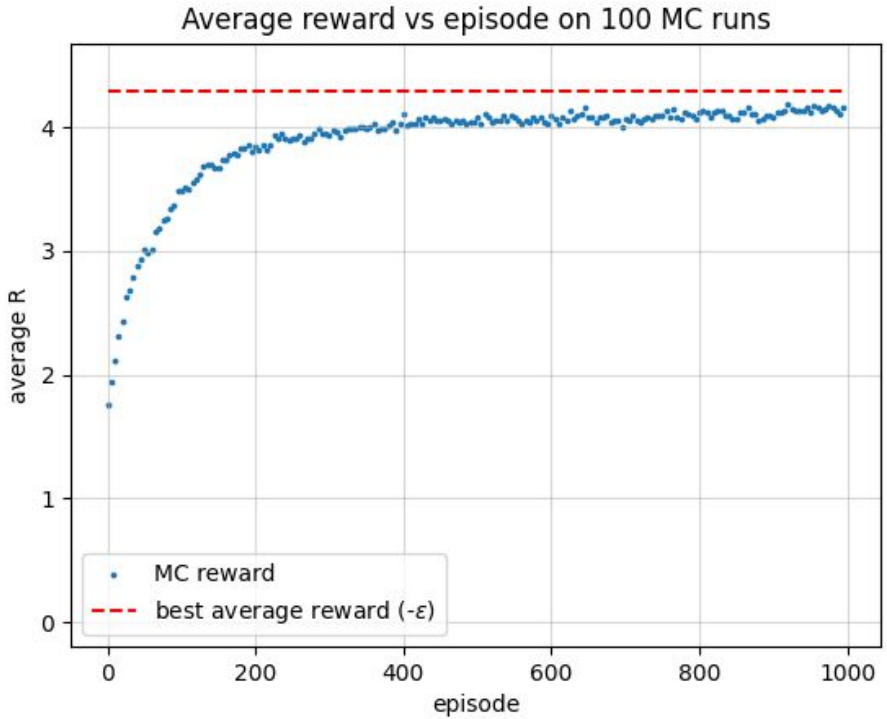


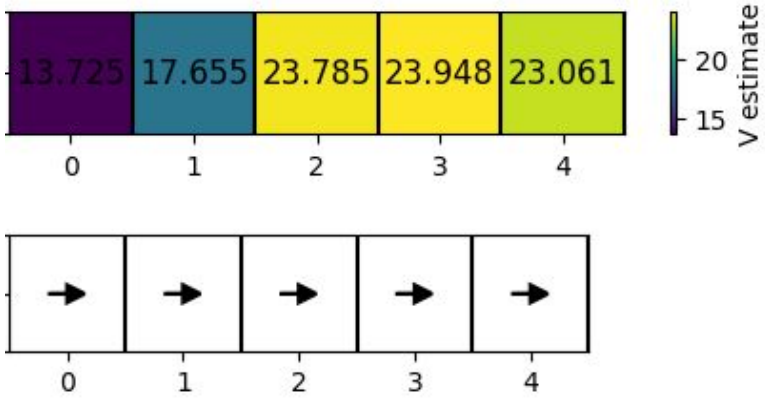

1. Random initialization of policy $\pi$ and state-action values $Q(s, a)$ for all $(s, a)$ pairs
2. Initialization of the episode (initial state) and ensurement that all actions have probability > 0 (ensure exploration)
3. Run a complete episode following $\pi$
4. Compute the returns as $G_t = R_t + G_{t-1}$ for $t = T - 1, \ldots, 0$ (reversing time saves computational resources)
5. (FIRST VISIT), for all $t$, if $(s_t, a_t)$ not in $[(s_{t-1}, a_{t-1}), \ldots, (s_0, a_0))$, append $G$ to returns of $(s_t, a_t)$ and update $Q(s_t, a_t) =< returns(s_t, a_t) >$
6. Go back to 2.

# The chain problem: MC



We solved The Chain Problem with multiple classic algorithms.
First off with **Monte Carlo**. We show results averaging 100 runs of 1000 episodes

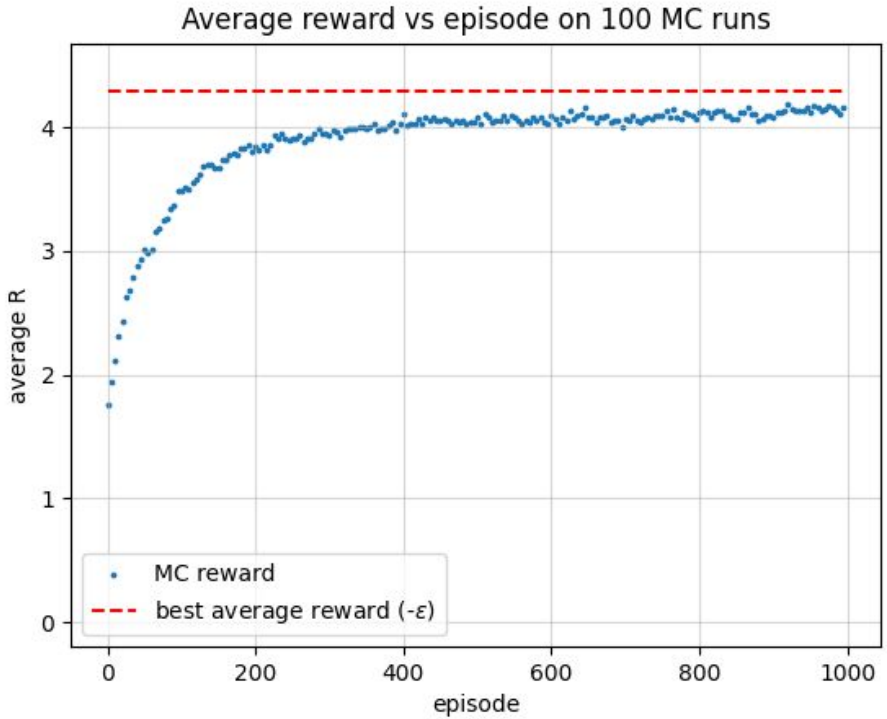


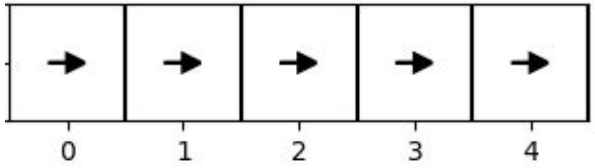
# The chain problem: MC



We solved The Chain Problem with multiple classic algorithms:
First off with **Monte Carlo**. We show results averaging 100 runs of 1000 episodes



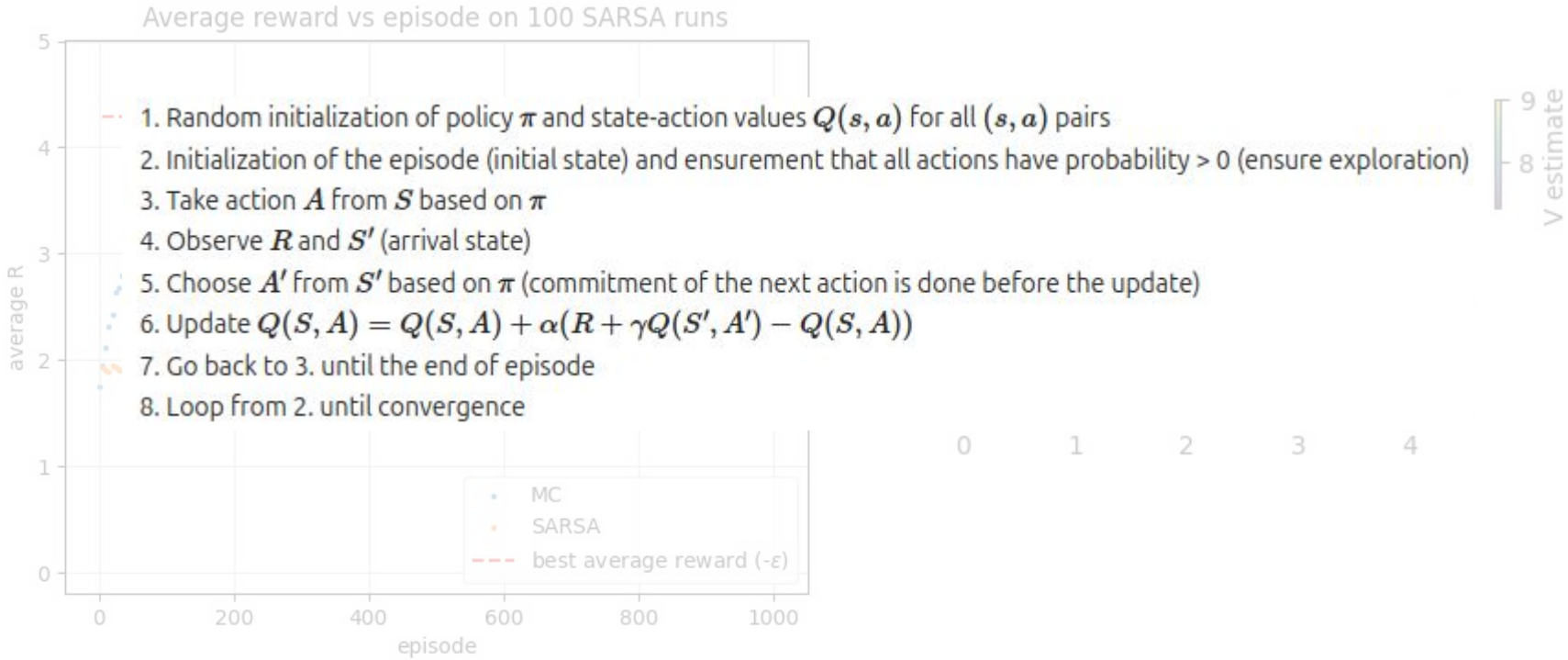

MC achieves the best strategy, but takes some time to do so



# The chain problem: SARSA



We tried also a **TD Learning** approach: **SARSA**, that implements updates immediately. We show results averaging 100 runs of 1000 episodes

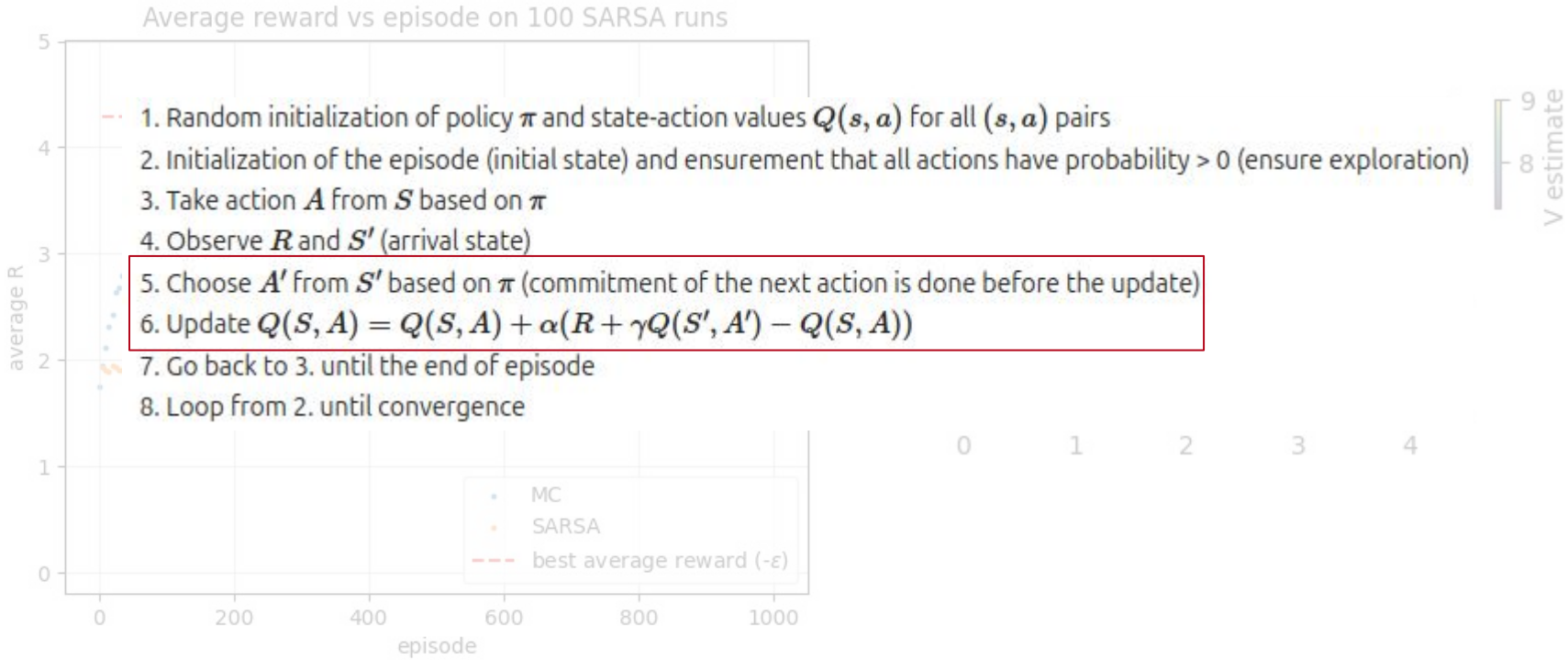



1. Random initialization of policy $\pi$ and state-action values $Q(s,a)$ for all $(s,a)$ pairs
2. Initialization of the episode (initial state) and ensurement that all actions have probability > 0 (ensure exploration)
3. Take action $A$ from $S$ based on $\pi$
4. Observe $R$ and $S'$ (arrival state)
5. Choose $A'$ from $S'$ based on $\pi$ (commitment of the next action is done before the update)
6. Update $Q(S,A) = Q(S,A) + \alpha(R + \gamma Q(S',A') - Q(S,A))$
7. Go back to 3. until the end of episode
8. Loop from 2. until convergence

# The chain problem: SARSA



We tried also a **TD Learning** approach: **SARSA**, that implements updates immediately.
We show results averaging 100 runs of 1000 episodes

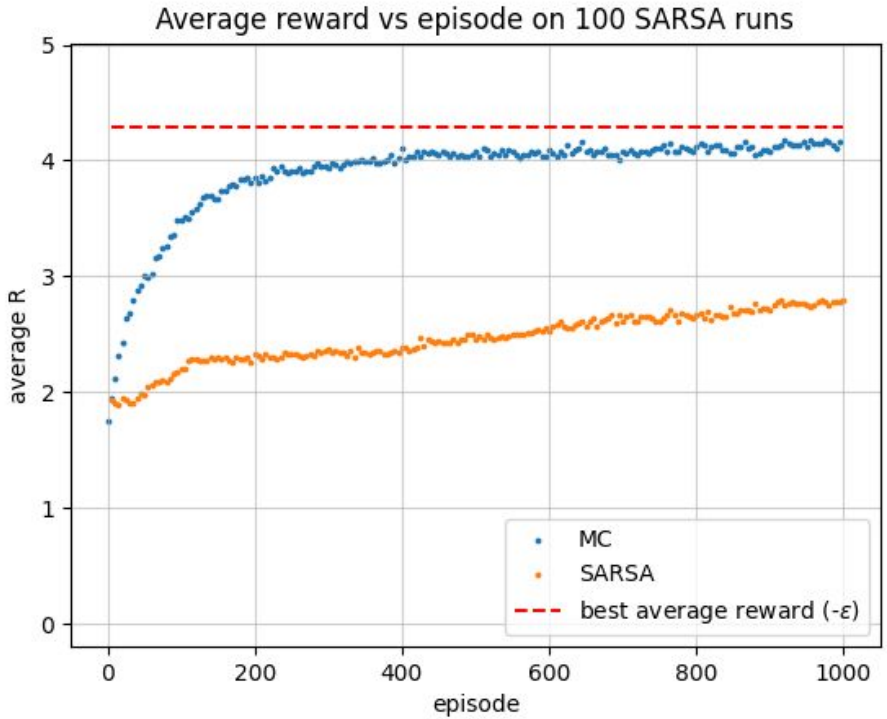


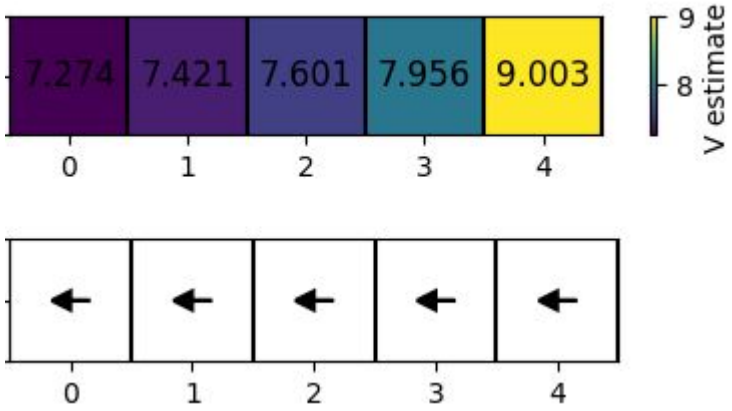

1. Random initialization of policy $\pi$ and state-action values $Q(s,a)$ for all $(s,a)$ pairs
2. Initialization of the episode (initial state) and ensurement that all actions have probability > 0 (ensure exploration)
3. Take action $A$ from $S$ based on $\pi$
4. Observe $R$ and $S'$ (arrival state)
5. Choose $A'$ from $S'$ based on $\pi$ (commitment of the next action is done before the update)
6. Update $Q(S,A) = Q(S,A) + \alpha(R + \gamma Q(S',A') - Q(S,A))$
7. Go back to 3. until the end of episode
8. Loop from 2. until convergence

# The chain problem: SARSA



We tried also a **TD Learning** approach: **SARSA**, that implements updates immediately.
We show results averaging 100 runs of 1000 episodes

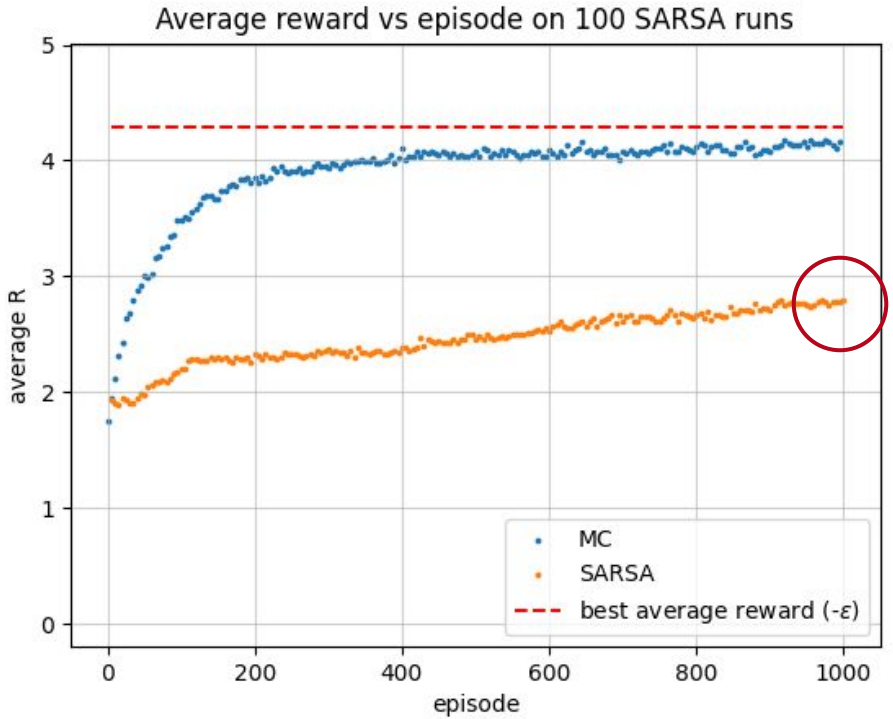


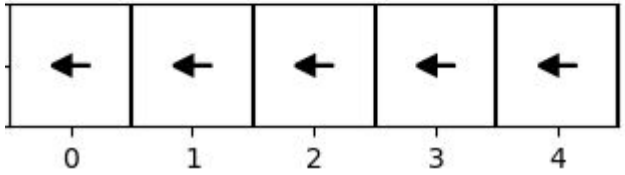
# The chain problem: SARSA



We tried also a **TD Learning** approach: **SARSA**, that implements updates immediately.
We show results averaging 100 runs of 1000 episodes



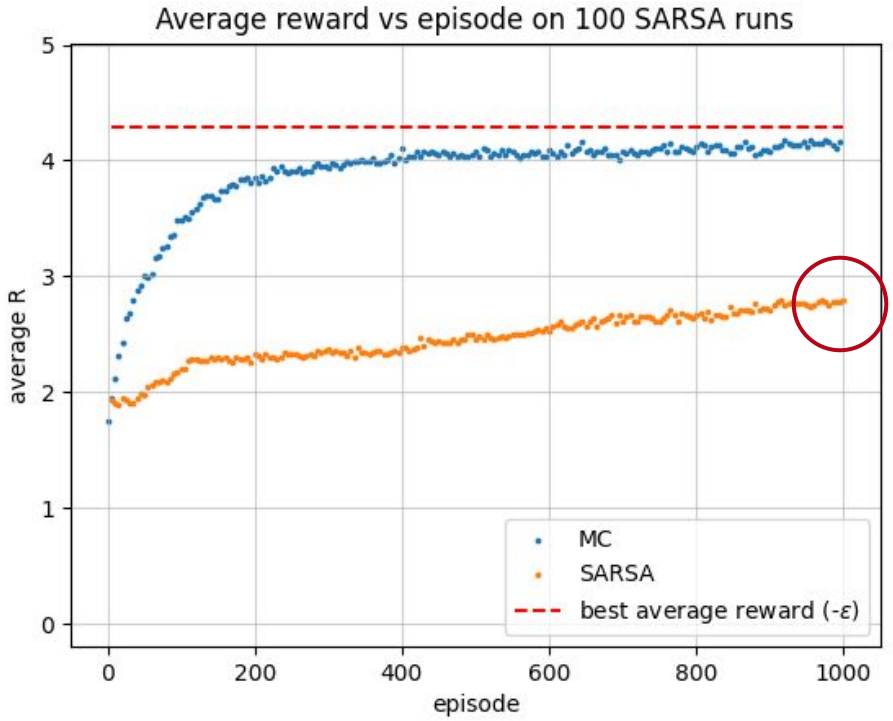

SARSA chooses the safe strategy rather than the most rewarding
The immediate V update has as consequence the preference towards quicker reward gaining, even if the value is lower

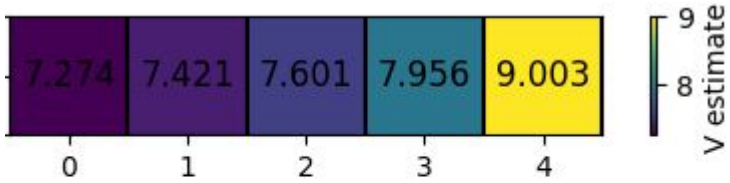


# The chain problem: SARSA



We tried also a **TD Learning** approach: **SARSA**, that implements updates immediately.
We show results averaging 100 runs of 1000 episodes




States all have similar values since the action "left" always resets to state 0

State 4 has higher value since there is a chance to get higher rewards

# Posterior Sampling RL



**Posterior Sampling** is a Bayesian approach to RL introduced by Strens in 2000
Is the extension of Thompson Sampling to **Markov Decision Processes** (MDPs).

# Posterior Sampling RL



**Posterior Sampling** is a Bayesian approach to RL introduced by Strens in 2000
Is the extension of Thompson Sampling to **Markov Decision Processes** (MDPs).

# Posterior Sampling RL



**Posterior Sampling** is a Bayesian approach to RL introduced by Strens in 2000
Is the extension of Thompson Sampling to **Markov Decision Processes** (MDPs).

We define MDPs as tuples: $M = (S, A, P, R, H)$

The sampling is performed over entire environment dynamics

# Posterior Sampling RL



**Posterior Sampling** is a Bayesian approach to RL introduced by Strens in 2000
Is the extension of Thompson Sampling to **Markov Decision Processes** (MDPs).

We define MDPs as tuples: $M = (S, A, P, R, H)$
The sampling is performed over entire environment dynamics

The goal is to find the policy that maximizes the expected sum of rewards

# Posterior Sampling RL



**Posterior Sampling** is a Bayesian approach to RL introduced by Strens in 2000
Is the extension of Thompson Sampling to **Markov Decision Processes** (MDPs).

We define MDPs as tuples: *M = (S, A, P, R, H)*
The sampling is performed over entire environment dynamics

The goal is to find the policy that maximizes the expected sum of rewards

Let $P = (T(s_i, a_j, s'_k))$ be the transition probability vector from any state-action pair *(s,a)* to a new state *s'*. We assume them to be independent. **Dirichlet** priors are used for state-action pairs since is conjugate to multinomial (that is the likelihood)

# Posterior Sampling RL



Dirichlet density: $p(P) \sim \prod_{s'} P(s'|s,a)^{\alpha_{s,a}(s')-1}$

Likelihood for one transition
(probability of such transition given the model): $L(P) = P(s'|s,a)$

or given the number of transitions $N(s,a,s')$ : $L(P(\cdot|s,a)) = \prod_{s'} P(s'|s,a)^{N(s,a,s')}$

From Bayes' theorem we get the posterior:

$$\prod_{s'} P(s'|s,a)^{\alpha_{s,a}(s')-1+N(s,a,s')} \sim Dir(\alpha_{s,a} + N(s,a,\cdot))$$

that is Dirichlet again!

# Posterior Sampling RL



Dirichlet density: $p(P) \sim \prod_{s'} P(s'|s,a)^{\alpha_{s,a}(s')-1}$

Likelihood for one transition
(probability of such transition given the model): $L(P) = P(s'|s,a)$

or given the number of transitions $N(s,a,s')$ : $L(P(\cdot|s,a)) = \prod_{s'} P(s'|s,a)^{N(s,a,s')}$

From Bayes' theorem we get the posterior:

$$\prod_{s'} P(s'|s,a)^{\alpha_{s,a}(s')-1+N(s,a,s')} \sim Dir(\alpha_{s,a} + N(s,a,\cdot))$$

that is Dirichlet again!

# Posterior Sampling RL



**Posterior Sampling algorithm:**

1) Sample the MDP $P(\cdot|s,a) \sim Dir(\alpha_{s,a})$
2) Compute optimal value function and optimal state-action value function:

$$V_t^k(s) = max_a Q_t^k(s,a) \qquad Q_t^k(s,a) = R(s,a) + \sum_{s'} P_k(s'|s,a) V_{t+1}^k(s')$$

3) Determine the greedy policy and execute it for an episode

$$\pi_k(s) = argmax_a Q^k(s,a)$$

4) Update the posterior and repeat for any visited *(s,a,s')* tuple:

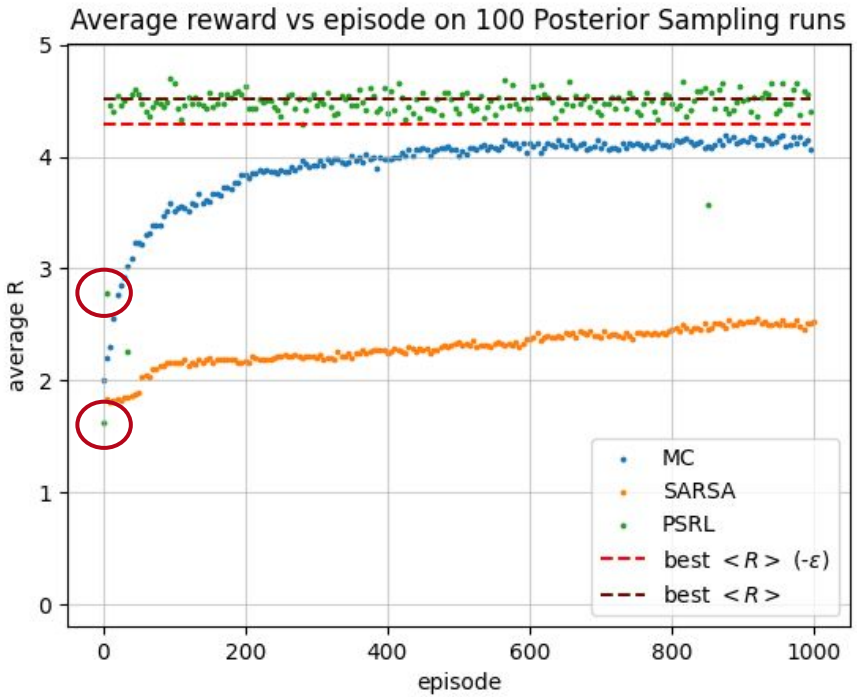
$$\alpha_{s,a}(s') \leftarrow \alpha_{s,a}(s') + 1$$

The algorithm then implements **deep** and **coherent** exploration for every policy π it runs an episode with. This leads the agent to find best rewards that are hidden behind **long exploration paths**

# The chain problem: PSRL

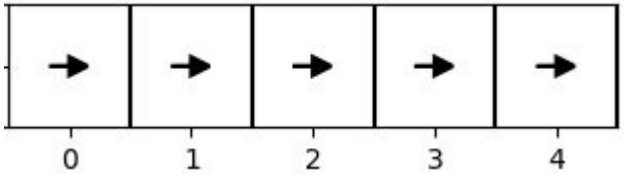


Finally we try to implement **Posterior Sampling RL** via a Dirichlet Prior:




With Posterior Sampling we achieve the best strategy after just few episodes:
we have a minimization of the regret and the best performance with minimal data

PSRL needs just an episode to reach the best reward to set it as its main strategy



# Application 3: Slippery Gridworld

# Problem introduction: Gridworld



The gridworld is another classic RL problem: it involves exploring a grid and finding the exit in the least number of steps possible. In particular we have that:

- The **state space** is the ensemble of all the cells the agent can find itself in
- The **action space** is A={up, down, left, right}. All actions are always available regardless of the state
- If an action moves the agent in a **prohibited state**, this gets a **reward of -100** and is teleported back to the **starting state**
- Each step yields a **reward** of -1, hence the agent is encouraged to take the least amount possible, without falling into forbidden states

# Problem introduction: Gridworld



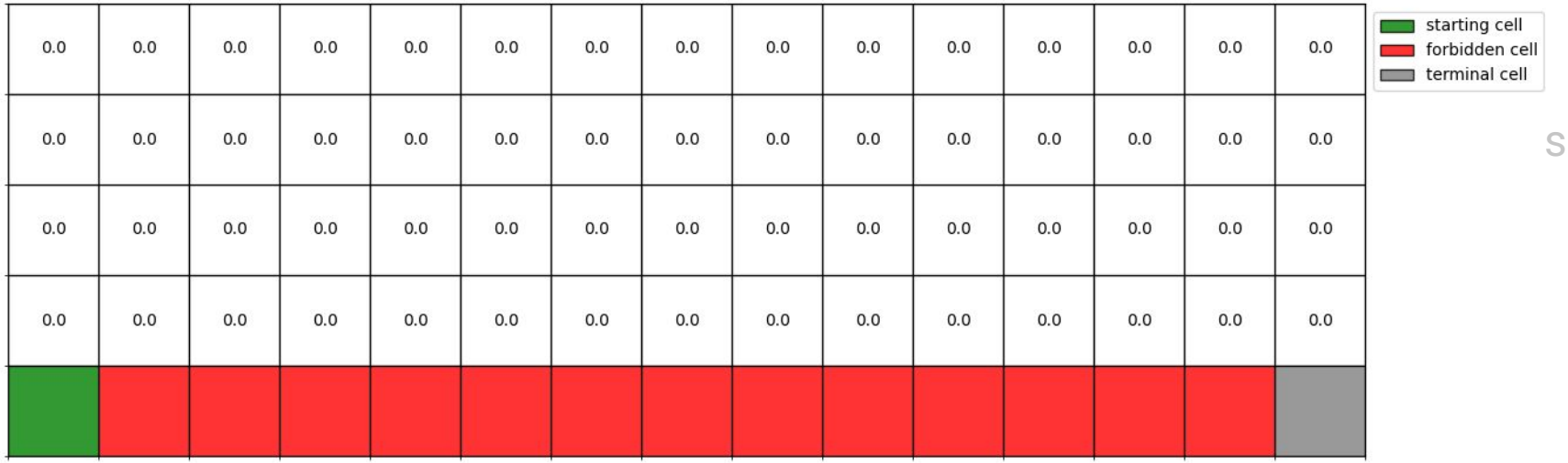
The gridworld is another classic RL problem. It involves exploring a grid and finding the exit in the least number of steps possible.

The **state space** is the ensemble of all the cells the agent can find himself in

The
reg

If a
as

Eac
am

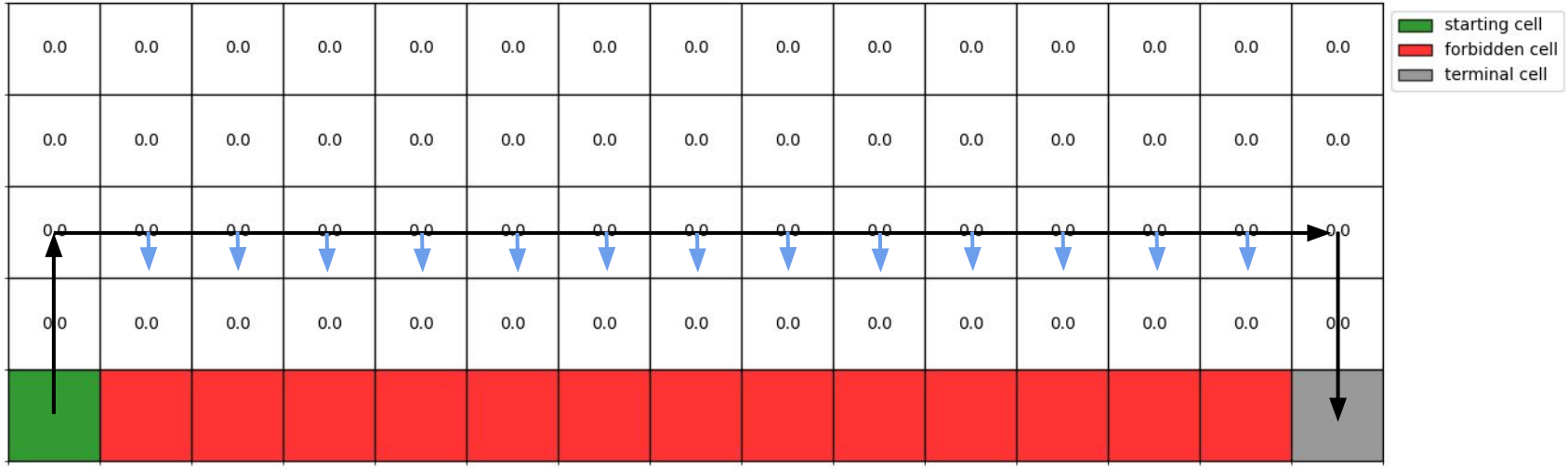



s

# Problem introduction: Slippery GW



The agent must find the shortest path to the exit, but **there is a catch**!
Regardless of the chosen action, the agent has always a small chance to “slip” (move down): the best strategy is a **tradeoff between the shortest and the safest path**.



# Slippery GW: classical solutions



Any problem with known **MDP** can be solved by the use of **Bellman Equations**. With these we can provide an estimate of a solved Q(s,a) implementing **Dynamic Programming**

R=-1 for every step  R=-100 if forbidden  γ=0.8  slip prob = 0.1

| | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| -4.91 | -4.89 | -4.86 | -4.82 | -4.78 | -4.73 | -4.66 | -4.57 | -4.46 | -4.33 | -4.16 | -3.95 | -3.69 | -3.36 | -2.95 |
| -4.89 | -4.86 | -4.82 | -4.78 | -4.73 | -4.66 | -4.57 | -4.46 | -4.33 | -4.16 | -3.95 | -3.69 | -3.36 | -2.95 | -2.44 |
| -4.86 | -4.82 | -4.78 | -4.73 | -4.66 | -4.57 | -4.46 | -4.33 | -4.16 | -3.95 | -3.69 | -3.36 | -2.95 | -2.44 | -1.8 |
| -4.82 | -4.78 | -4.73 | -4.66 | -4.57 | -4.46 | -4.33 | -4.16 | -3.95 | -3.69 | -3.36 | -2.95 | -2.44 | -1.8 | -1.0 |
| | | | | | | | | | | | | | | |

starting cell
forbidden cell
terminal cell

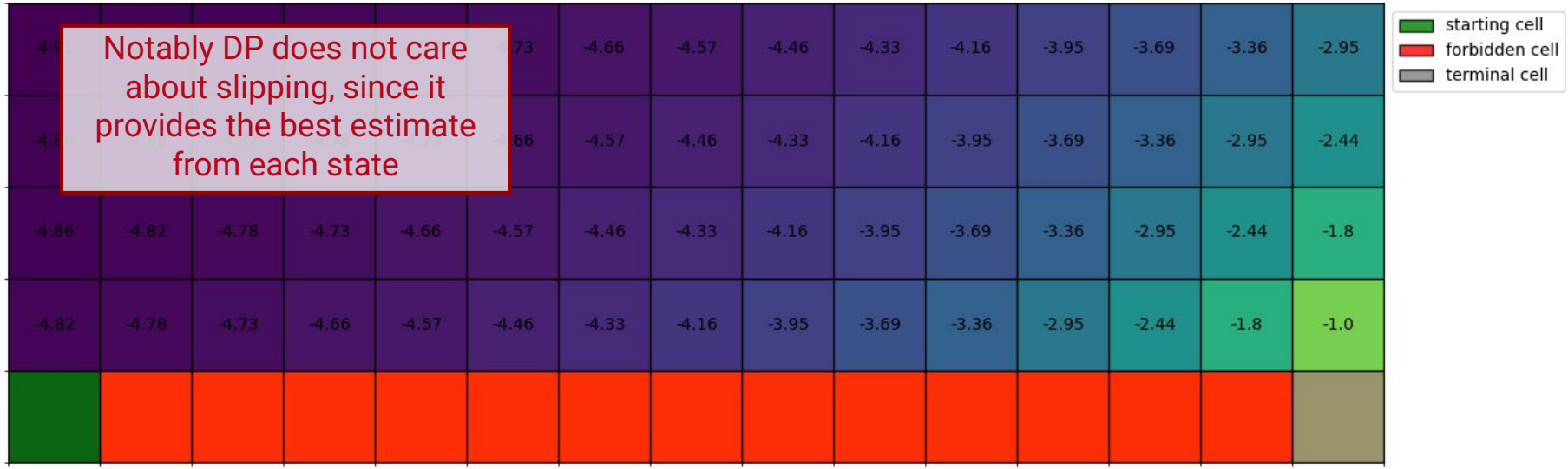
# Slippery GW: classical solutions



Any problem with known **MDP** can be solved by the use of **Bellman Equations**. With these we can provide an estimate of a solved Q(s,a) implementing **Dynamic Programming**

R=-1 for every step  R=-100 if forbidden  γ=0.8  slip prob = 0.1

Notably DP does not care about slipping, since it provides the best estimate from each state

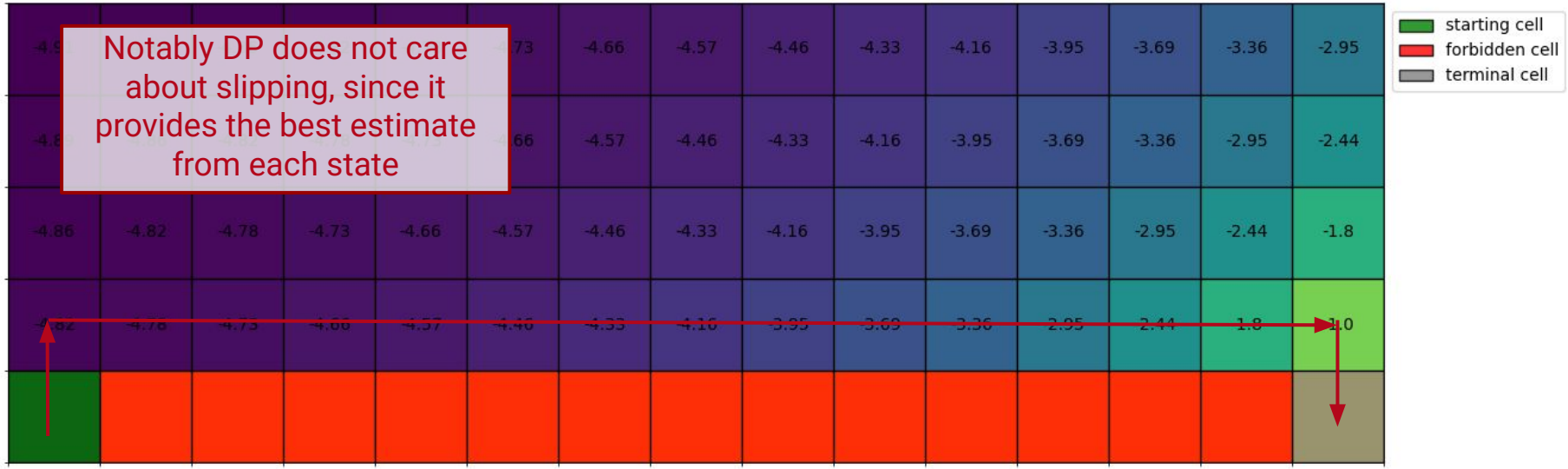


# Slippery GW: classical solutions



Any problem with known **MDP** can be solved by the use of **Bellman Equations**. With these we can provide an estimate of a solved Q(s,a) implementing **Dynamic Programming**

R=-1 for every step  R=-100 if forbidden  γ=0.8  slip prob = 0.1



# Slippery GW: Monte Carlo



We tried to solve the Slippery GW with classic RL algorithms: **Monte Carlo**
We trained all the algorithms with **2500 episodes** and evaluated them over **100**

R=-1 for every step  R=-100 if forbidden  γ=0.8  slip prob = 0.1

| | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| -4.993 | -4.983 | -4.978 | -4.996 | -5.127 | -4.813 | -4.747 | -4.659 | -4.558 | -4.43 | -4.251 | -4.029 | -3.721 | -3.225 | -3.736 |
| -4.995 | -4.997 | -4.959 | -4.978 | -5.053 | -4.876 | -4.804 | -4.752 | -4.674 | -4.825 | -4.67 | -4.301 | -4.061 | -2.687 | -2.031 |
| -4.974 | -4.967 | -4.941 | -4.931 | -4.907 | -4.874 | -4.853 | -4.916 | -4.774 | -4.992 | -5.0 | 0.0 | -4.27 | -2.092 | -1.023 |
| -4.992 | -4.993 | -4.991 | -4.977 | -5.263 | -6.268 | -4.874 | -5.639 | -5.417 | -5.521 | -5.374 | -5.191 | -1.8 | -1.068 | 0.0 |

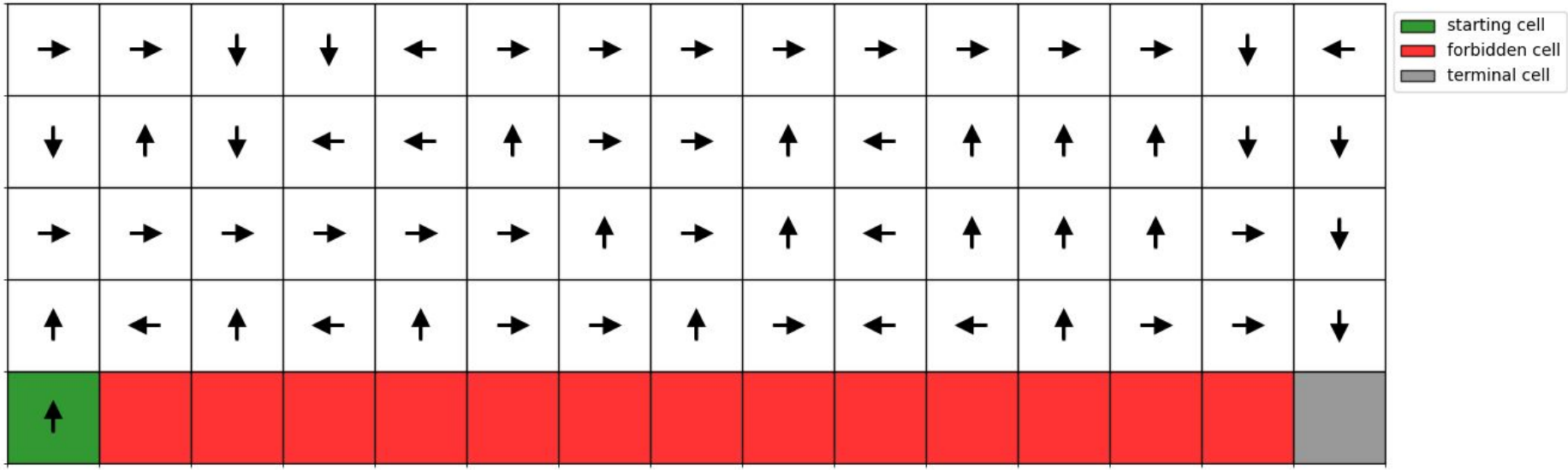
starting cell
forbidden cell
terminal cell

# Slippery GW: Monte Carlo



We tried to solve the Slippery GW with classic RL algorithms: **Monte Carlo**
We trained all the algorithms with **2500 episodes** and evaluated them over **100**

We observe that MC likes to follow a “safe path” to avoid slipping. The path is not optimal, but effective

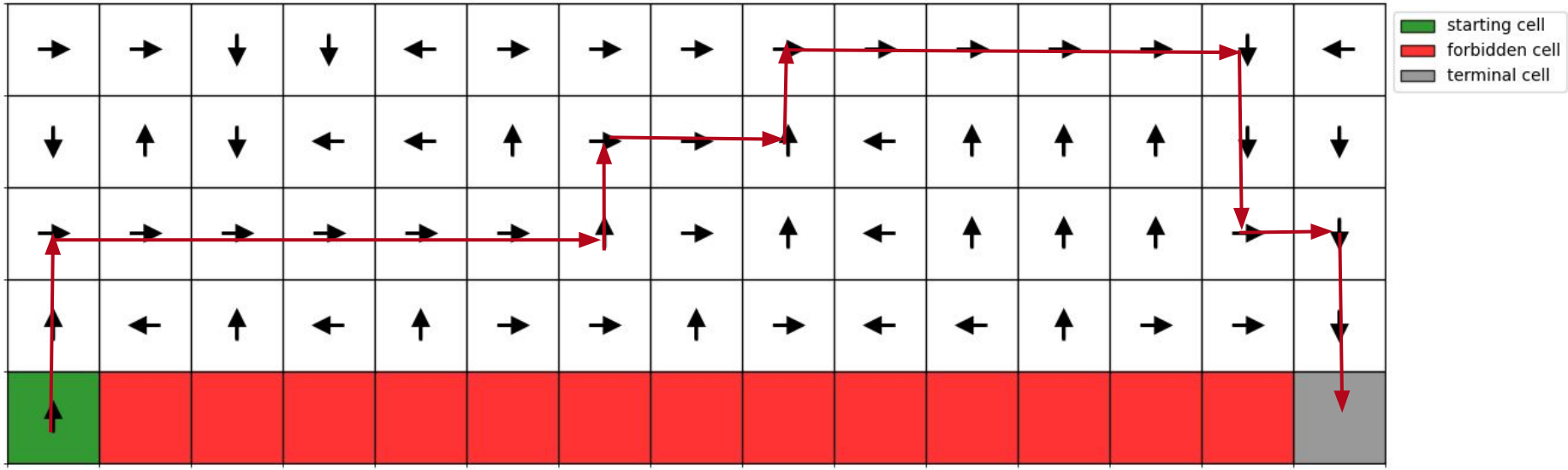


# Slippery GW: Monte Carlo



We tried to solve the Slippery GW with classic RL algorithms: **Monte Carlo**
We trained all the algorithms with **2500 episodes** and evaluated them over **100**

We observe that MC likes to follow a “safe path” to avoid slipping. The path is not optimal, but effective

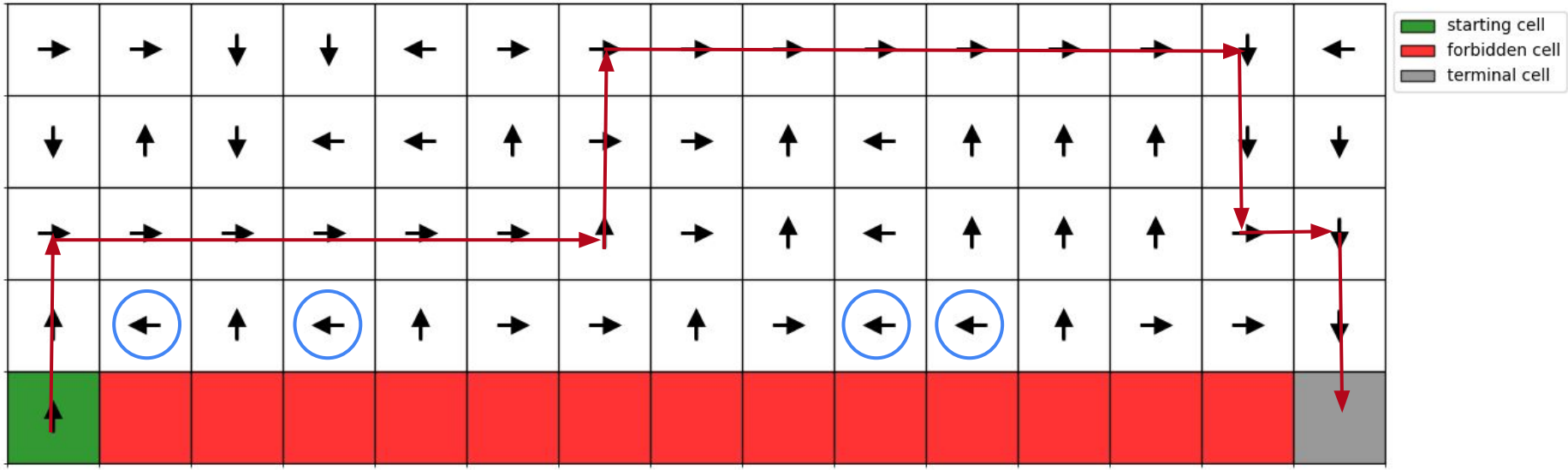


# Slippery GW: Monte Carlo



We tried to solve the Slippery GW with classic RL algorithms: **Monte Carlo**
We trained all the algorithms with **2500 episodes** and evaluated them over **100**

MC policy is resilient to fall into the forbidden area, but is not efficient in getting the lowest steps count



# Slippery GW: SARSA



We tried to solve the Slippery GW with classic RL algorithms: **SARSA**
We trained all the algorithms with **2500 episodes** and evaluated them over **100**

R=-1 for every step  R=-100 if forbidden  γ=0.8

| | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| -4.935 | -4.917 | -4.892 | -4.863 | -4.82 | -4.761 | -4.695 | -4.619 | -4.502 | -4.382 | -4.204 | -3.973 | -3.653 | -3.256 | -2.739 |
| -4.927 | -4.908 | -4.879 | -4.844 | -4.793 | -4.729 | -4.643 | -4.55 | -4.408 | -4.227 | -3.948 | -3.699 | -3.3 | -2.726 | -2.211 |
| -4.945 | -4.921 | -4.906 | -4.886 | -4.839 | -4.689 | -4.632 | -4.514 | -4.397 | -4.122 | -3.778 | -3.369 | -2.714 | -2.03 | -1.238 |
| -4.957 | -4.9 | -4.884 | -4.687 | -4.57 | -4.527 | -4.65 | -4.355 | -4.076 | -4.148 | -4.009 | -3.321 | -3.292 | -1.256 | 0.0 |
| | | | | | | | | | | | | | | |

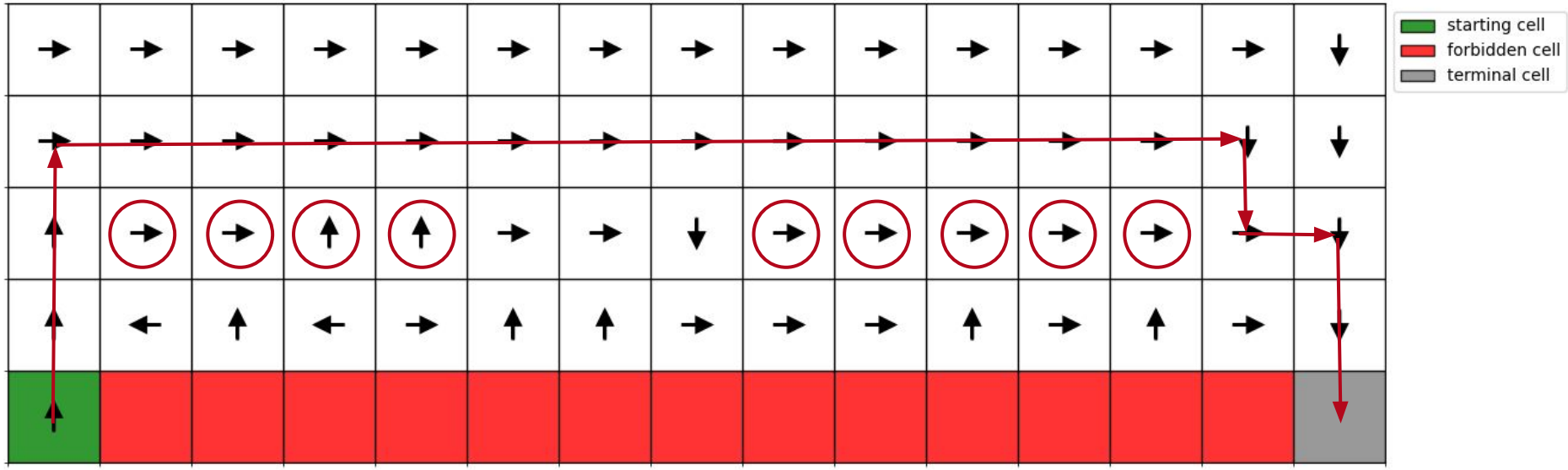
starting cell
forbidden cell
terminal cell

# Slippery GW: SARSA



We tried to solve the Slippery GW with classic RL algorithms: **SARSA**
We trained all the algorithms with **2500 episodes** and evaluated them over **100**

SARSA leads to a similar policy as MC
but is more resilient to a “slip” step



# Slippery GW: Posterior Sampling RL



Finally we attempted an implementation of **Posterior Sampling RL**
We trained the algorithm with **2500 episodes** and evaluated them over **100**

R=-1 for every step  R=-100 if forbidden  γ=0.8  slip prob = 0.1

| | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| -4.935 | -4.895 | -4.866 | -4.826 | -4.782 | -4.728 | -4.659 | -4.574 | -4.467 | -4.333 | -4.165 | -3.956 | -3.699 | -4.037 | -3.385 |
| -4.932 | -4.912 | -4.889 | -4.861 | -4.826 | -4.828 | -4.782 | -4.731 | -4.736 | -4.618 | -4.566 | -3.693 | -3.365 | -3.118 | -2.464 |
| -4.943 | -4.929 | -4.911 | -4.889 | -4.889 | -4.905 | -4.843 | -4.831 | -4.865 | -4.792 | -4.751 | -4.259 | -2.956 | -2.463 | -1.81 |
| -4.954 | -4.943 | -4.954 | -4.938 | -4.941 | -4.942 | -4.911 | -4.91 | -4.925 | -4.929 | -4.874 | -4.257 | -2.444 | -1.803 | -1.003 |
| | | | | | | | | | | | | | | |

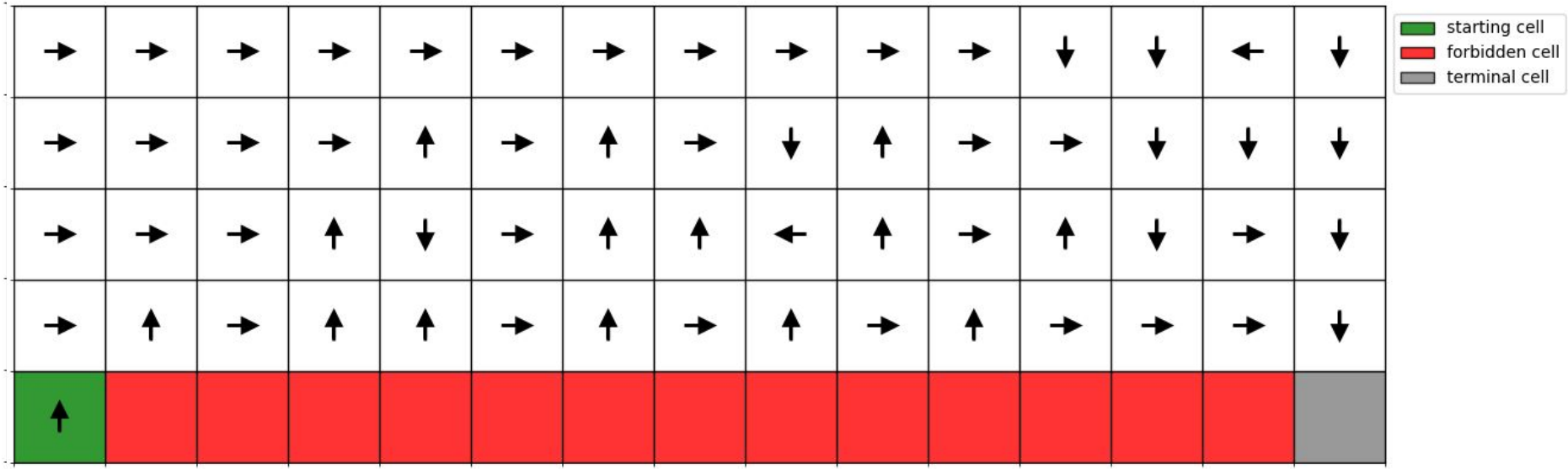
starting cell
forbidden cell
terminal cell

# Slippery GW: Posterior Sampling RL



Finally we attempted an implementation of **Posterior Sampling RL**
We trained the algorithm with **2500 episodes** and evaluated them over **100**

Posterior sampling is more conservative than the other strategies

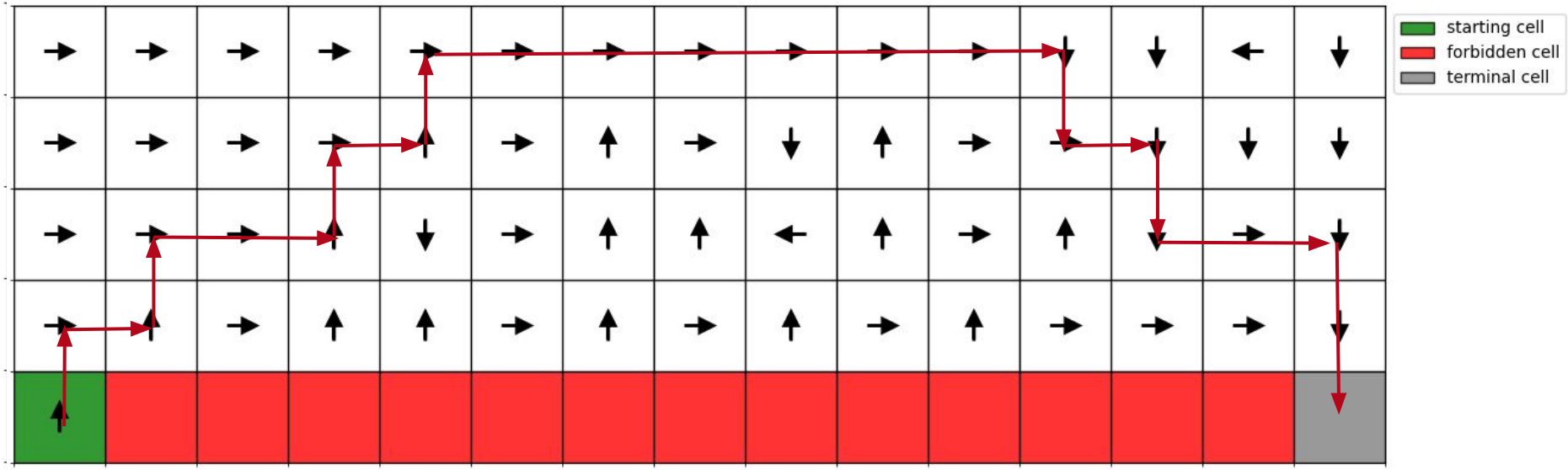


# Slippery GW: Posterior Sampling RL



Finally we attempted an implementation of **Posterior Sampling RL**
We trained the algorithm with **2500 episodes** and evaluated them over **100**

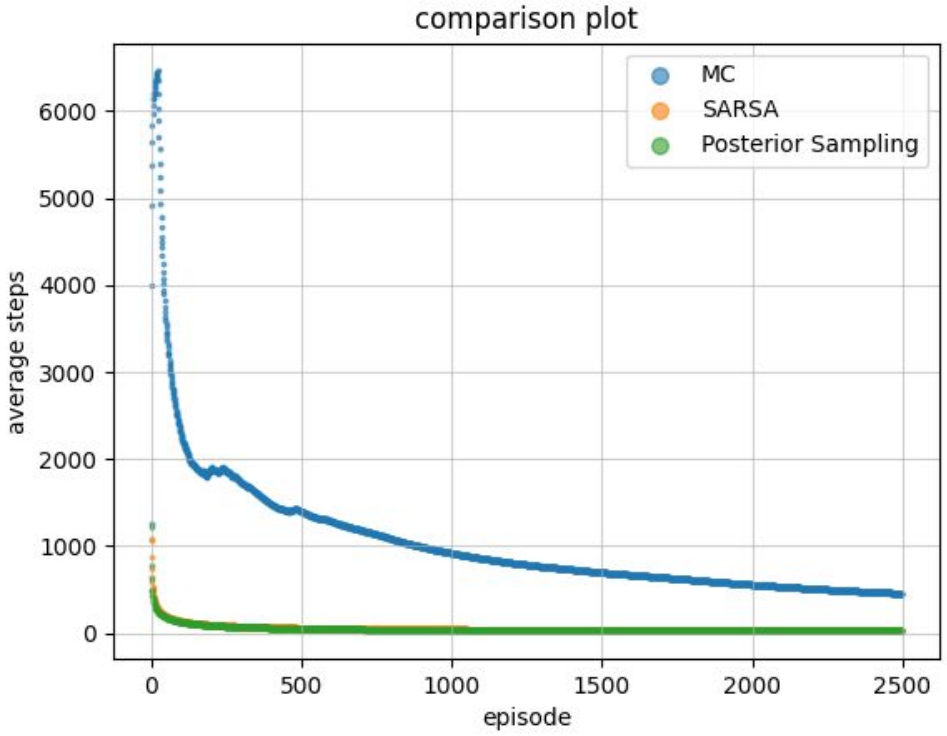
Posterior sampling is more conservative than the other strategies



# Slippery GW: comparison



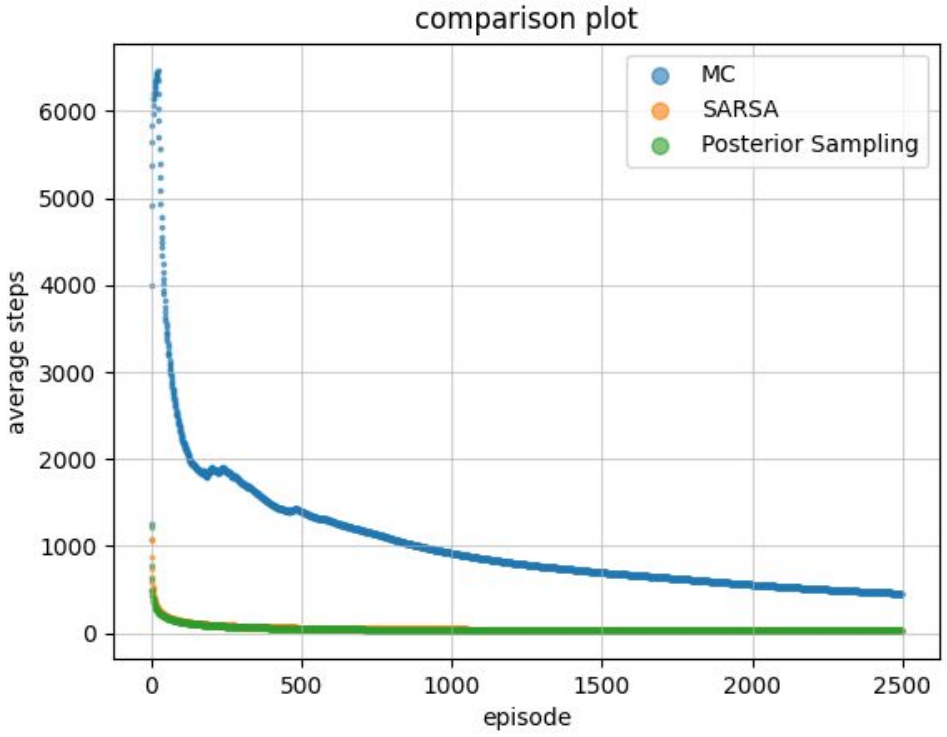
The performance improves **faster** during training of SARSA and PSRL

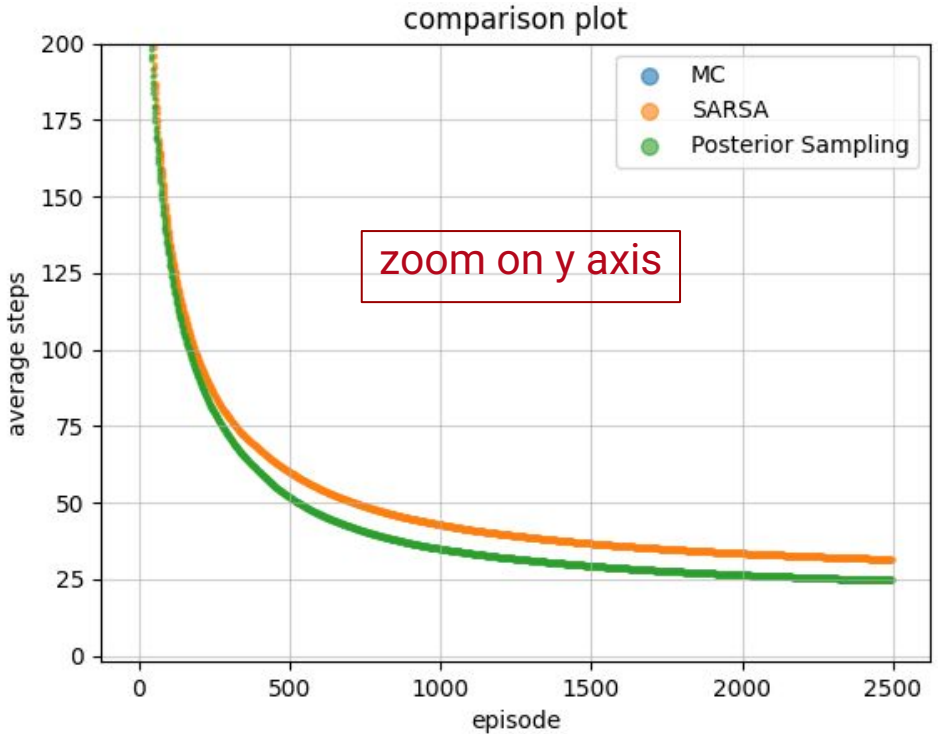


# Slippery GW: comparison



The performance improves **faster** during training of SARSA and PSRL

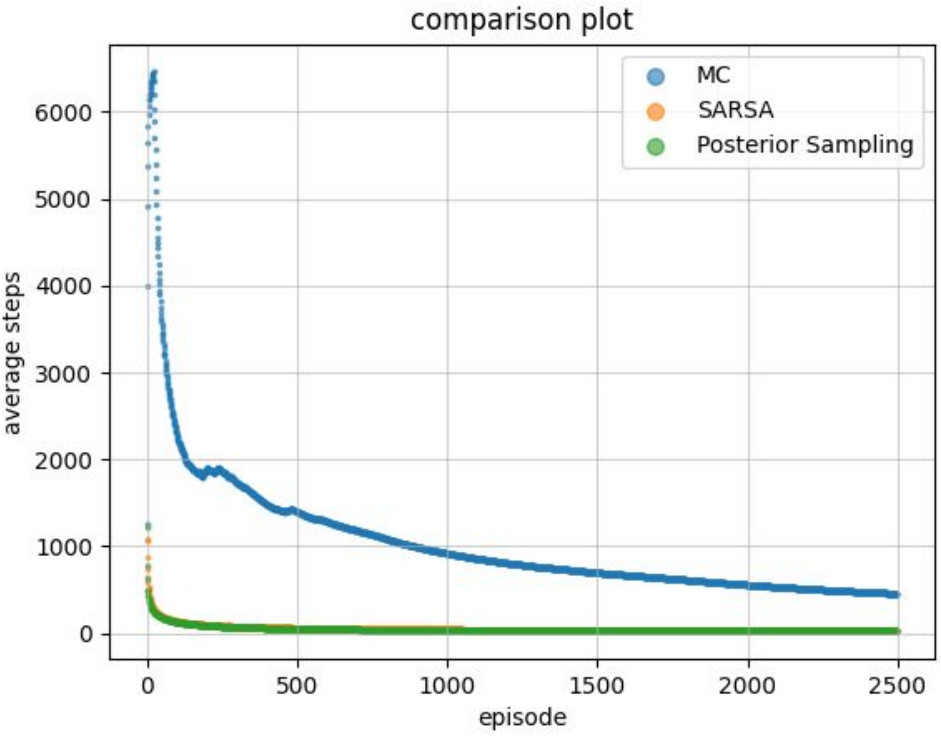


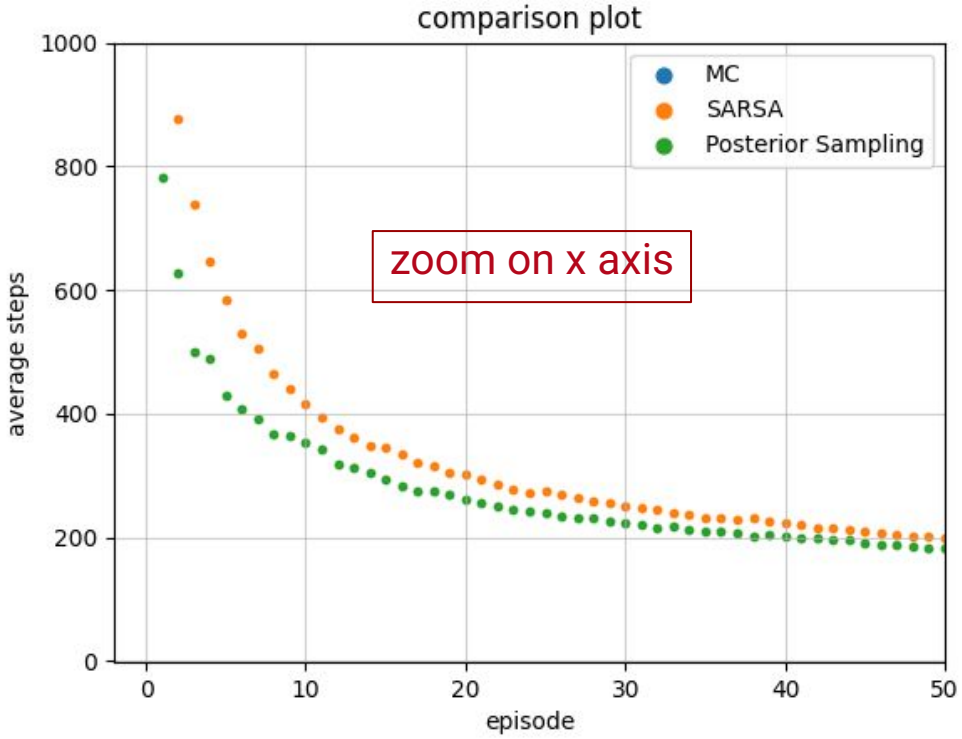



# Slippery GW: comparison



The performance improves **faster** during training of SARSA and PSRL

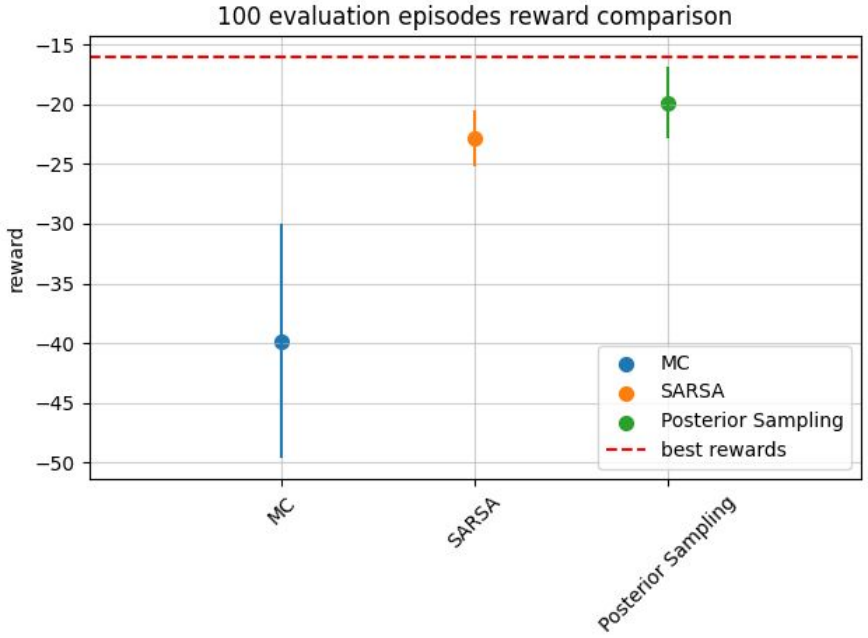


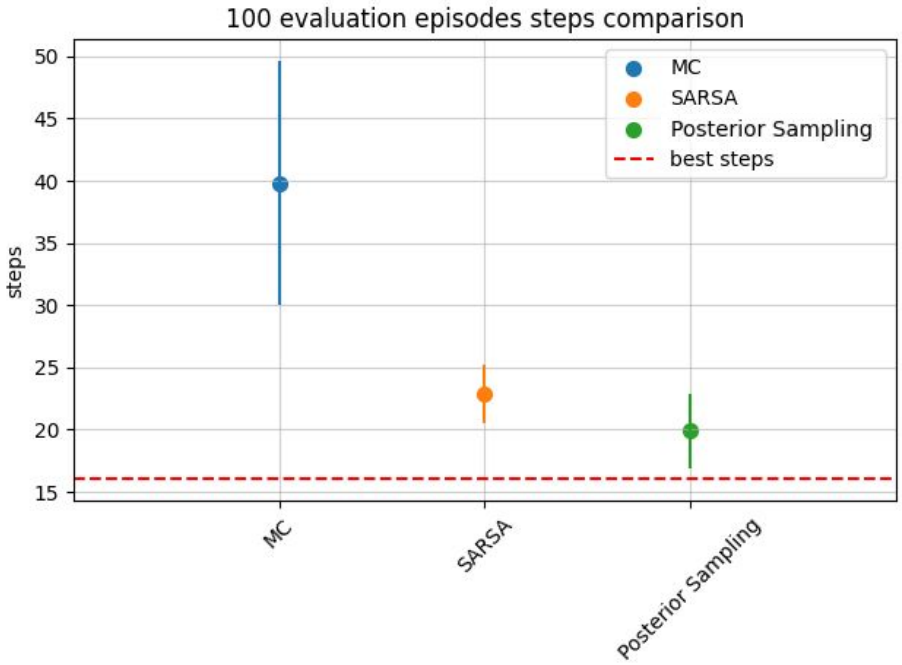



# Slippery GW: comparison



We evaluated the performance of obtained policies running **100** more episodes
We saved average rewards and steps count

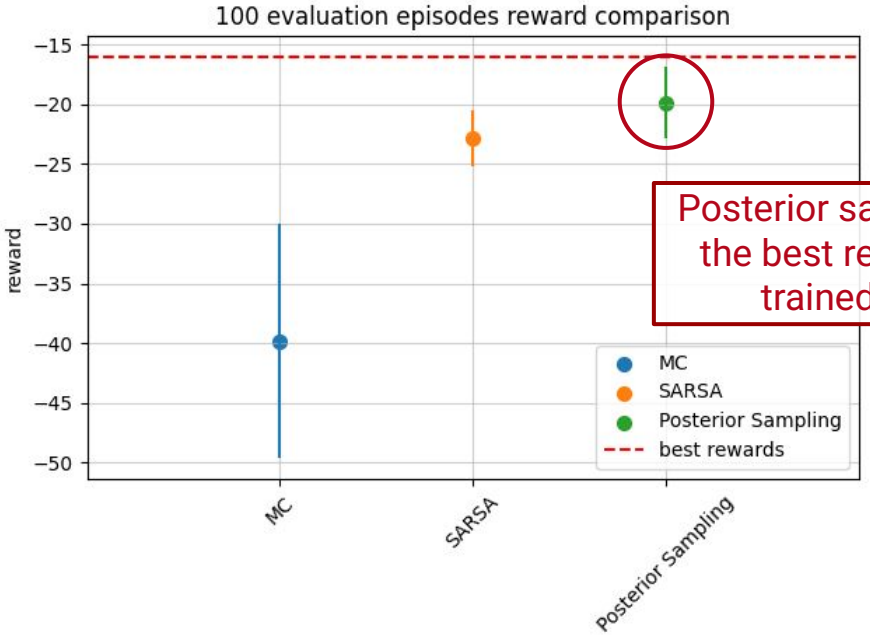


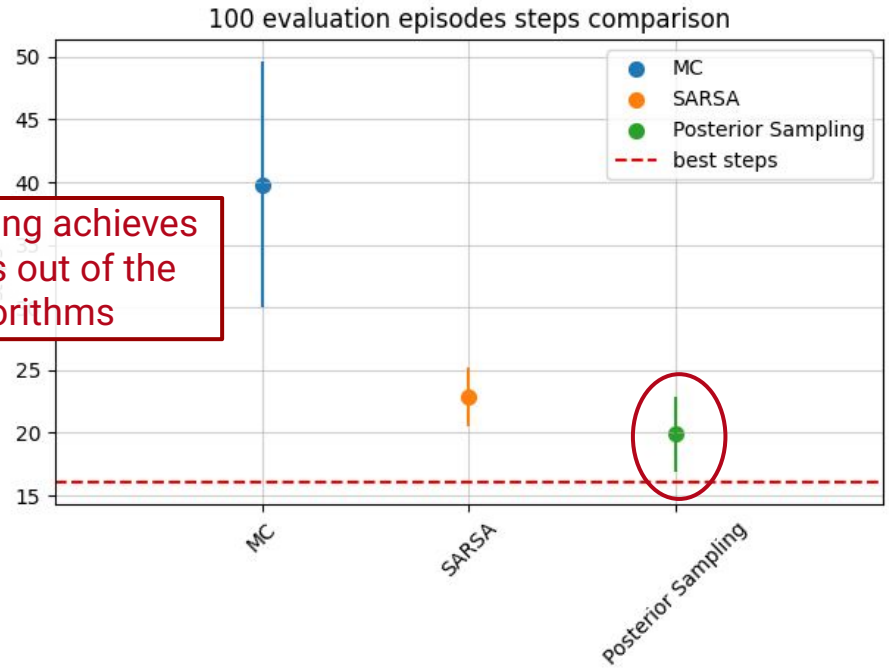



# Slippery GW: comparison



We evaluated the performance of obtained policies running **100** more episodes
We saved average rewards and steps count




Posterior sampling achieves the best results out of the trained algorithms

# Discussion



Bayesian RL has shown to be useful in all the presented problems:

| | General Bandits (mean Regret) | Chain Problem (mean Reward) | Gridworld (mean steps) | Gridworld (mean reward) |
|---|---|---|---|---|
| OIV / Monte Carlo | 0.167 ± 0.015 | 3.86 ± 0.38 | 39.71 ± 9.66 | -(3.3 ± 2.8)□$10^3$ |
| UCB / SARSA | 0.089 ± 0.013 | 2.16 ± 0.17 | 23.11 ± 1.80 | -22.90 ± 2.42 |
| VTS / PSRL | 0.071 ± 0.011 | 4.41 ± 0.54 | 19.88 ± 3.39 | -19.42 ± 2.72 |

The biggest improvements are obtained in the slippery gridworld.
**Bayesian Inference especially improves results in problems that punish naive exploration** and have a better **reward strategy hidden** behind **long exploration paths.**

# Conclusions



We can conclude that:

- it is feasible to set up a **Bayesian Inference** approach both in simple problems as Bandits (with **Variational Inference**) and in more complex ones (**Posterior Sampling**). More advanced problems can be tackled with state-of-the-art approaches [5]
- in Bandits, Bayesian approaches do not provide significant efficiency improvements compared to classical RL algorithms, but they can be useful to **avoid blind hyper-parameters tuning**.
- in more complex problems such as Gridworld and The Chain Problem, Bayesian Inference provides a significant improvement in performance, mostly in **early episodes**.

  In particular, we observe that **Bayesian approaches** are **consistent in achieving close-to-optimal efficiency** throughout all problems, unlike classical RL algorithms.

# Thank you!

# Bibliography



[1] M. Strens, *A Bayesian Framework for Reinforcement Learning*, International Conference on Machine Learning (ICML-2000), 2000
[2] M. Ghavamzadeh et al., *Bayesian Reinforcement Learning: A Survey*, Foundations and Trends in Machine Learning, now, 2015
[3] Urteaga, Iñigo & Wiggins, Chris. (2017). *Variational inference for the multi-armed contextual bandit*. 10.48550/arXiv.1709.03163
[4] Bishop, C. M., & Nasrabadi, N. M. (2006). *Pattern recognition and machine learning* (Vol. 4, No. 4, p.738). New York: springer
[5] Deisenroth M.P. & Rasmussen C.E. (2011). *PILCO: A Model-Based and Data-Efficient Approach to Policy Search*, International Conference on Machine Learning

# Memes

Monte Carlo
SARSA
Posterior Sampling

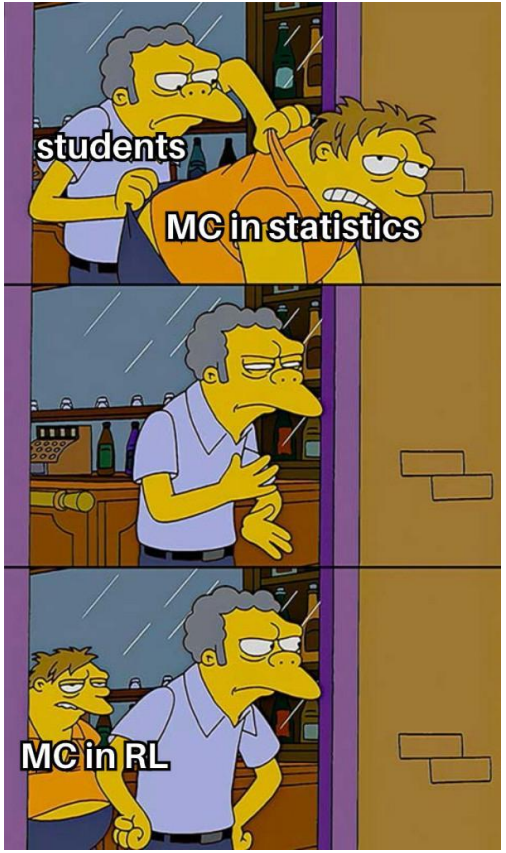
other problems having multiple states
Bandits

# Memes

SARSA
Posterior Sampling

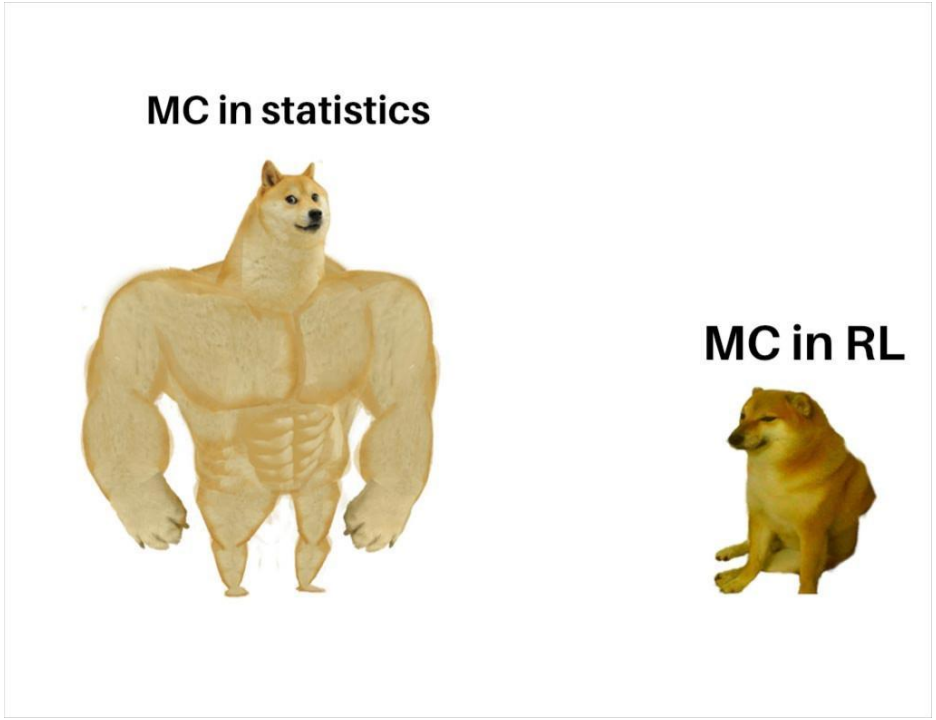
MC on Gridworld

No thanks, I use Monte Carlo methods


students
MC in statistics
MC in RL

# Memes





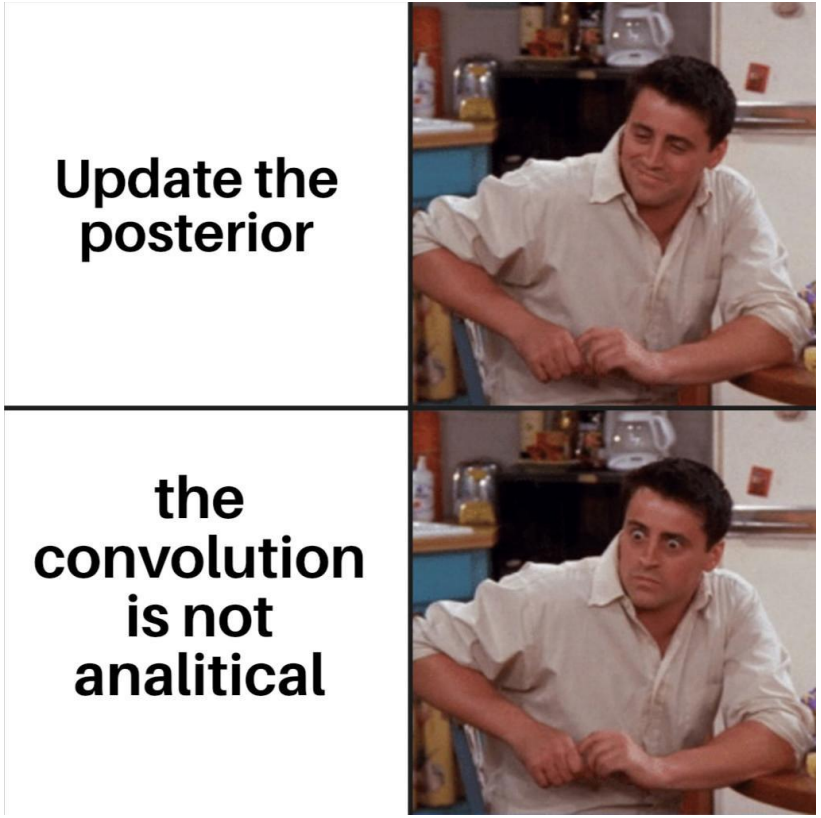
# Memes

A problem that does not punish fast updates
SARSA

RL is not about estimating posterior
BAYESIAN INFERENCE
BAYESIAN INFERENCE
BAYESIAN INFERENCE

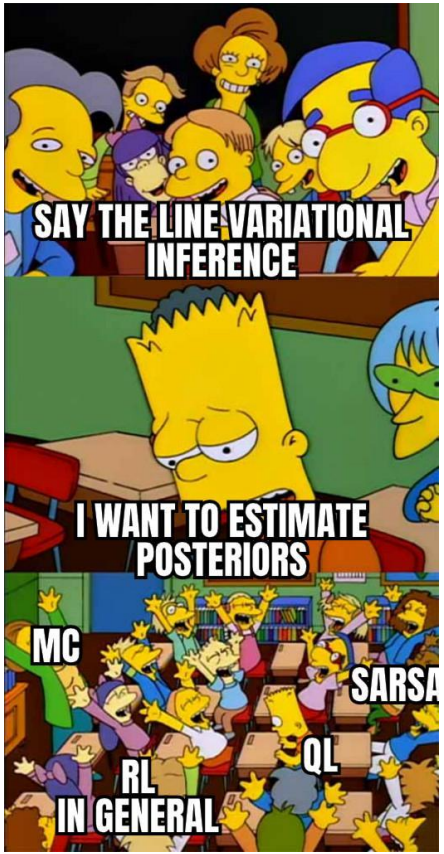
# Memes

Bayesian Inference
Woah.
RL DEVELOPERS
This is worthless!


Update the posterior
the convolution is not analitical

# Memes

Unsupervised problems
RL
ML
DL
ensemble models
regression


SAY THE LINE VARIATIONAL INFERENCE
I WANT TO ESTIMATE POSTERIORS
MC
SARSA
QL
RL IN GENERAL